Research Reference: StarForth Physics Runtime

Volume III of the StarshipOS Technical Reference

Robert A. James

Contents

1 Published Paper: Steady-State Convergence **9**
- 1.1 Citation . . . 9
- 1.2 Abstract . . . 9
- 1.3 Key Findings . . . 9
- 1.4 Executive Summary . . . 9
- 1.5 Executive Summary . . . 9
  - 1.5.1 The Problem . . . 9
  - 1.5.2 What Was Built . . . 9
  - 1.5.3 Experimental Results . . . 10
  - 1.5.4 Implications . . . 10
  - 1.5.5 Scope and Limitations . . . 11
  - 1.5.6 Reproducibility . . . 11
  - 1.5.7 Summary . . . 11
- 1.6 Campaign Overview Notes . . . 11
- 1.7 StarForth, LithosAnanke, and StarshipOS: A Technical Overview . . . 11
  - 1.7.1 StarForth . . . 11
  - 1.7.2 LithosAnanke . . . 13
  - 1.7.3 StarshipOS . . . 14
- 1.8 Peer Review Notes . . . 15

2 Mathematical Companion **17**
- 2.1 Q48.16 Fixed-Point Arithmetic . . . 17
- 2.2 Feedback Loop Equations . . . 17
- 2.3 Figures . . . 17

3 Formal Claims, Anti-Claims, and Null Hypothesis **19**
- 3.1 Formal Claim Table . . . 19
- 3.2 Formal Claim Table . . . 19
  - 3.2.1 Primary Claims . . . 19
  - 3.2.2 Claim C1: Algorithmic Determinism . . . 19
  - 3.2.3 Claim C2: Adaptive Convergence . . . 20
  - 3.2.4 Claim C3: Variance Separation . . . 21
  - 3.2.5 Claim C4: Reproducibility . . . 21
  - 3.2.6 Claim C5: Statistical Significance . . . 21
  - 3.2.7 Supporting Claims . . . 22
  - 3.2.8 Claim Dependency Structure . . . 22
  - 3.2.9 Patent-Relevant Claims . . . 22
  - 3.2.10 Using This Table in Review . . . 22
- 3.3 Anti-Claims . . . 23
- 3.4 Out-of-Scope Claims: What This Work Does Not Assert . . . 23
  - 3.4.1 Physics and Thermodynamics . . . 23

3.4.2 Optimality and Performance . . . 23
3.4.3 Machine Learning and Artificial Intelligence . . . 23
3.4.4 Novelty and Prior Art . . . 24
3.4.5 Formal Verification . . . 24
3.4.6 Causation and Interpretation . . . 24
3.4.7 Generalization . . . 24
3.4.8 Deployment Status . . . 25
3.4.9 Statistical Claims . . . 25
3.4.10 Scope Exclusions . . . 25
3.4.11 Summary Table . . . 25
3.4.12 How to Use This Section in Peer Review . . . 25
3.5 Null Hypothesis and Falsification Criteria . . . 26
3.6 Null Hypothesis and Falsification Framework . . . 26
3.6.1 Statement of the Null Hypothesis . . . 26
3.6.2 Evidence Against the Null Hypothesis . . . 26
3.6.3 What Would Falsify the Claims . . . 27
3.6.4 Null Model Predictions . . . 27
3.6.5 Bayesian Interpretation . . . 28
3.6.6 Control Experiments . . . 28
3.6.7 Statistical Power . . . 28
3.6.8 Alternative Explanations Considered . . . 28
3.6.9 Burden of Proof . . . 28
3.7 Negative Results . . . 29
3.8 Negative Results: Documented Failure Modes . . . 29
3.8.1 Workload-Dependent Failures . . . 29
3.8.2 Parameter-Dependent Failures . . . 29
3.8.3 Environmental Failures . . . 30
3.8.4 Architectural Failures . . . 30
3.8.5 Implementation Bugs Resolved . . . 30
3.8.6 Statistical Failures . . . 30
3.8.7 Generalization Failures . . . 31
3.8.8 Summary of Failure Modes . . . 31
3.8.9 Lessons . . . 31
3.9 Academic Wording Guidelines . . . 32
3.10 Academic Wording Guidelines . . . 32
3.10.1 The Core Rule: Similarity, Not Equivalence . . . 32
3.10.2 Approved Phrases . . . 32
3.10.3 Forbidden Phrases . . . 33
3.10.4 Before-and-After Examples . . . 34
3.10.5 Defensive Language Patterns . . . 34
3.10.6 Section-Specific Guidelines . . . 35
3.10.7 Reviewer Challenge Responses . . . 35
3.10.8 Pre-Publication Checklist . . . 35
3.10.9 The Golden Rule . . . 36

4 Reproducibility and Independent Replication 37
4.1 One-Command Reproduction . . . 37
4.2 Reproducibility Protocol . . . 37
4.2.1 The One-Command Reproduction . . . 37
4.2.2 Exact Reproduction Environment . . . 37
4.2.3 Reference Hardware . . . 38
4.2.4 Environment Configuration . . . 38

4.2.5 Full 90-Run Experiment . . . 38
4.2.6 Statistical Validation . . . 39
4.2.7 Absence of Hidden Randomness . . . 39
4.2.8 Cross-Platform Reproduction . . . 39
4.2.9 Troubleshooting . . . 39
4.2.10 Long-Term Archival . . . 39
4.2.11 Contact . . . 40
4.3 Replication Invitation . . . 40
4.4 Invitation to Independent Replication . . . 40
4.4.1 Why Replicate . . . 40
4.4.2 Replication Levels . . . 40
4.4.3 Exact Reproduction via Docker . . . 41
4.4.4 Manual Environment Configuration . . . 41
4.4.5 Reporting Results . . . 41
4.4.6 Common Issues and Resolutions . . . 42
4.4.7 Acknowledgments for Falsification . . . 42
4.4.8 Cross-Institutional Collaboration . . . 42
4.4.9 Replication Checklist . . . 42
4.5 Scientific Defense Checklist . . . 43
4.6 Scientific Defense Checklist . . . 43
4.6.1 Attack Vector 1: Data Integrity . . . 43
4.6.2 Attack Vector 2: Statistical Validity . . . 43
4.6.3 Attack Vector 3: Experimental Design . . . 44
4.6.4 Attack Vector 4: Claims versus Evidence . . . 44
4.6.5 Attack Vector 5: Reproducibility . . . 44
4.6.6 Attack Vector 6: Prior Art and Novelty . . . 44
4.6.7 Attack Vector 7: Implementation Correctness . . . 45
4.6.8 Attack Vector 8: Generalization . . . 45
4.6.9 Attack Vector 9: Conflict of Interest . . . 45
4.6.10 Attack Vector 10: Over-Interpretation . . . 46
4.6.11 Defense Maturity Matrix . . . 46
4.7 Sensitivity Analysis Design . . . 46
4.8 Sensitivity Analysis: Parameter Robustness . . . 46
4.8.1 Purpose . . . 46
4.8.2 Parameters Under Test . . . 47
4.8.3 Window Size Sensitivity . . . 47
4.8.4 Cache Size Sensitivity . . . 47
4.8.5 Decay Coefficient Sensitivity . . . 48
4.8.6 Heartbeat Period Sensitivity . . . 48
4.8.7 ANOVA Significance Level Sensitivity . . . 49
4.8.8 Multi-Parameter Sweep . . . 49
4.8.9 Catastrophic Parameter Values . . . 49
4.8.10 Boundary Summary . . . 49
4.8.11 Robustness Metrics . . . 50
4.8.12 Summary . . . 50
4.9 Physics Optimization Reproduce . . . 50
4.10 Reproducing the Hot-Words Cache Experiment . . . 50
4.10.1 Prerequisites . . . 50
4.10.2 Build . . . 51
4.10.3 Running the Benchmark . . . 51
4.10.4 Interpreting Key Metrics . . . 51

4.10.5 Q48.16 Fixed-Point Arithmetic . . . 51
4.10.6 Diagnostic Checklist . . . 51
4.10.7 References . . . 52
4.11 Peer Review Defense . . . 52
4.12 Peer Review Defence — Pre-Submission Record . . . 52
4.12.1 Objection 1: "Is 0% Variance Realistic?" . . . 52
4.12.2 Objection 2: "Is the 25% Improvement Just Warmup?" . . . 52
4.12.3 Objection 3: "How Can You Claim Stability with 70% Variance?" . . . 52
4.12.4 Objection 4: "Why Trust Bounds from Only 30 Runs?" . . . 52
4.12.5 Objection 5: "This is Just a Toy Example (Fibonacci)" . . . 52
4.12.6 Anticipated Positive Question: ML-Based Optimisation . . . 53

5 Roadmap **55**
5.1 Timeline Summary . . . 55
5.2 LithosAnanke Roadmap . . . 55
5.3 StarshipOS . . . 55
5.4 FPGA / Custom Silicon . . . 55

6 Ontology of Thermodynamic Terms **57**
6.1 Ontology, Taxonomy, and Lexicon . . . 57
6.1.1 Conceptual Framework . . . 57
6.1.2 Taxonomy . . . 58
6.1.3 Lexicon . . . 58
6.1.4 Avoided and Deprecated Terms . . . 60
6.1.5 Mathematical Formalism . . . 60
6.1.6 Ontological Commitments . . . 60
6.1.7 Usage Guidelines for Academic Writing . . . 61
6.1.8 References . . . 61

7 Related Work **63**
7.1 Literature Review: Physics-Driven VM Optimisation . . . 63
7.1.1 VM Optimisation Techniques . . . 63
7.1.2 Real-Time Metrics-Driven Systems . . . 63
7.1.3 Physics-Inspired and Biologically-Inspired Computing . . . 64
7.1.4 Formal Verification of VM Optimisation . . . 64
7.1.5 Stack-Based and FORTH Virtual Machines . . . 64
7.1.6 Microkernel-Compatible Systems . . . 64
7.1.7 The Novelty Gap . . . 65
7.1.8 Recommended Citations . . . 65
7.1.9 Positioning Statement . . . 65
7.2 Published Results . . . 65
7.3 Results for Publication: Physics-Driven VM Optimisation . . . 65
7.3.1 Abstract (150 words, PLDI/ASPLOS format) . . . 65
7.3.2 Experimental Setup . . . 66
7.3.3 Core Results . . . 66
7.3.4 Publication-Ready Tables . . . 67
7.3.5 Statistical Rigour . . . 67
7.3.6 Claims . . . 67
7.3.7 Reproducibility Statement . . . 68
7.4 Research Outline . . . 68
7.5 Research Paper Outline: Physics-Driven VM Optimisation . . . 68
7.5.1 Working Title . . . 68

7.5.2 Target Format and Scope . . . 68
7.5.3 Section Structure . . . 68
7.5.4 Key Figures . . . 70
7.5.5 Key Tables . . . 70

8 James Law Experimental Protocol **71**
8.1 James Law — Window Scaling Experiment Protocol . . . 71
8.1.1 Objective . . . 71
8.1.2 Conservation Invariant . . . 71
8.1.3 Experimental Design . . . 71
8.1.4 Key Metrics . . . 72
8.1.5 Collapse Threshold Detection . . . 72
8.1.6 Success Criteria . . . 72
8.1.7 Planned Extensions . . . 72

A Glossary **73**

Glossary of Terms **75**
A.1 Research Archive Notes . . . 79
A.2 Research Directory Overview . . . 79
A.2.1 Key Research Contributions . . . 79
A.2.2 Publication Materials . . . 79
A.2.3 Experimental Reproducibility . . . 80

Chapter 1

Published Paper: Steady-State Convergence

1.1 Citation

The primary experimental results are published in:

?

The paper is available at `docs/working/papers/published/James_Steady-State_Convergence_Adapt`

1.2 Abstract

1.3 Key Findings

1.4 Executive Summary

1.5 Executive Summary

1.5.1 The Problem

Modern runtime systems face a persistent conflict between two design goals. Adaptive systems—contemporary web browsers, mobile application runtimes, JIT-compiled virtual machines—learn which code paths run most often and reorganize themselves to accelerate those paths. They improve over time, but their internal decisions vary from run to run; two executions of the same program on the same machine may produce different optimization states. This non-determinism is acceptable for web browsers but disqualifying for safety-critical software, financial systems requiring auditable behavior, or real-time systems with timing guarantees.

Deterministic systems—firmware for medical devices, aircraft flight controllers, certified real-time kernels—run the same way on every invocation. They can be formally verified, exhaustively tested, and certified by standards bodies. But they cannot adapt: the optimization configuration frozen at compile time may be far from optimal for the actual workload seen at runtime.

StarForth resolves this conflict. The system is simultaneously adaptive and completely predictable at the level of its algorithmic decisions.

1.5.2 What Was Built

StarForth is a FORTH-79 compliant virtual machine with a physics-driven adaptive runtime. Using execution frequency as a proxy for thermal energy (a deliberate conceptual metaphor, not

a physics claim), the system tracks how often each dictionary entry executes, applies exponential decay to reduce the weight of stale executions, and maintains a small cache of the most frequently executed words for accelerated lookup. Every decision in this pipeline is deterministic: given the same execution sequence, the system produces the same cache configuration on every run.

Seven feedback loops coordinate the adaptive behavior:

- Execution heat tracking and hot-words caching (Loop 1)
- Rolling window of execution history (Loop 2)
- Linear decay of quiescent entries (Loop 3)
- Word-to-word transition prediction (Loop 4)
- Variance-based window width inference via Levene's test (Loop 5)
- Decay slope inference via exponential regression (Loop 6)
- Adaptive heartbeat coordination (Loop 7)

All loops are independently togglable and formally characterized. Fixed-point arithmetic ($\mathbb{Q}_{48.16}$ throughout) eliminates IEEE-754 non-determinism across architectures.

1.5.3 Experimental Results

A three-configuration design of experiments ran 90 trials (30 runs per configuration):

- **C_NONE** (baseline): all adaptive loops disabled
- **C_CACHE**: hot-words cache enabled, inference disabled
- **C_FULL**: all seven loops active

Result 1: algorithmic determinism. Across all 30 runs of C_FULL, the cache hit rate was 17.39% with a coefficient of variation (CV) of exactly 0.00%. The F-test for variance homogeneity yields $F(29, 29) \to \infty$, $p < 10^{-30}$. Even under intentional thermal stress (sustained CPU load), the algorithmic decisions did not change; only wall-clock runtime increased.

Result 2: adaptive convergence. C_FULL improved from a mean of 10.20 ms in early runs (1–15) to 7.61 ms in late runs (16–30), a statistically significant 25.4% improvement ($t = 4.23$, $p < 0.001$, Cohen's $d \approx 5.08$). The baseline configuration (C_NONE) showed slight degradation over the same window; C_CACHE showed no meaningful improvement. Only the fully adaptive configuration converged.

Result 3: variance decomposition. Runtime exhibits 60–70% CV due to OS scheduler noise, thermal variation, and cache-line effects. Cache decisions exhibit 0.00% CV. The two components are statistically independent (Pearson $r = 0.03$, $p = 0.87$). The adaptive algorithm is perfectly stable; the environment adds noise on top of a deterministic foundation.

1.5.4 Implications

Verifiable adaptive systems. A system whose adaptive decisions are deterministic can, in principle, be formally verified. The optimization state reachable from a given workload is predictable, auditable, and reproducible—properties required for safety-critical certification.

Reproducible performance characterization. Benchmark results become portable: different researchers running the same workload reach the same adaptive steady state, enabling apples-to-apples comparison of optimization strategies.

Adaptation without non-determinism. The results demonstrate that statistical inference can guide optimization without introducing randomness. The system adapts through arithmetic and counting, not machine learning or stochastic search.

1.5.5 Scope and Limitations

This work does not claim that StarForth is the fastest virtual machine, that its approach is superior to JIT compilation for all use cases, or that the thermodynamic framework is a physical theory. The 0.00% algorithmic CV applies to cache decisions, not to total system runtime. The system is validated on CPU-bound, deterministic FORTH programs; I/O-bound workloads, programs with random control flow, and very short processes fall outside the validated scope. Fifteen failure modes are documented in the companion negative-results section.

1.5.6 Reproducibility

Full source code, raw experimental data from all 90 runs, and a step-by-step reproduction protocol are publicly available. A Docker container provides bit-for-bit exact reproduction against a pinned environment. The authoritative reproduction command is:

```
git clone https://github.com/rajames440/StarForth.git
cd StarForth
make fastest
./build/amd64/fastest/starforth --doe
```

Deviations from the expected 0.00% algorithmic CV should be reported as bugs.

1.5.7 Summary

- An adaptive virtual machine can achieve 0.00% algorithmic variance across 90 experimental runs.
- Adaptation and determinism are not mutually exclusive; statistical inference drives optimization without introducing non-determinism.
- Performance improves measurably (25.4%) as the system converges to steady state.
- All data, code, and reproduction instructions are publicly available for independent verification.

1.6 Campaign Overview Notes

1.7 StarForth, LithosAnanke, and StarshipOS: A Technical Overview

Author: Robert A. James. Date: 9 April 2026. Status: Working Document.

1.7.1 StarForth

What It Is

StarForth is a pure C99 FORTH-79 virtual machine with no libc dependencies. It is self-adaptive: it observes its own execution behavior and continuously adjusts internal parameters to maintain a conservation invariant called K≡1.0. It runs on x86_64, aarch64, and RISC-V.

Why FORTH

FORTH is correct for this work for reasons beyond familiarity. FORTH words are discrete, classifiable, observable computational units — each word has a definite stack-effect signature, each word consumes a known quantity of computational energy, each word is a particle with measurable properties. This is the foundation of StarForth's self-adaptive behavior.

The Primitive Taxonomy

StarForth implements 23 word modules (18 FORTH-79 standard, 5 StarForth extensions). Every primitive word carries quantum-analog properties derived from its stack-effect signature:

Spin. Derived from net stack-effect arithmetic. A word consuming two values and producing one has spin $-\frac{1}{2}$; these values fall directly from `( n n - n )` notation, as half-integer spin falls from the Dirac equation.

Charge. The net stack delta. Charge conservation across a complete program means the program is stack-balanced.

Heat. Execution frequency and recency. Heat decays over time, analogous to radioactive decay.

Mass. Bytecode footprint of the word definition.

Color. Binding state: white words are fully resolved; colored words carry unresolved forward references.

Three of these quantities — spin, charge, and color — behave as genuine conservation laws across well-formed programs. This is a structural property of the system discovered empirically and subsequently formalized in Isabelle/HOL.

The SSM and James Law

The SSM (Steady State Machine) is the self-adaptive core of StarForth. It monitors execution behavior through instrumented subsystems and continuously adjusts internal parameters to maintain K≡1.0.

James Law ($K \equiv 1.0$) is a conservation invariant: the normalized total execution energy of the StarForth universe is conserved at unity across all configurations and workloads. Validation basis: 38,400 experimental runs spanning 128 factorial parameter configurations, CV below 1%, published on SSRN (abstract 5930134).

The SSM operates through four mechanisms:

Rolling Window of Truth (RWOT). A sliding observation window encoding five observables: word identity, prev→word transition, instruction pointer position, execution heat, and causal stack depth. Updated at heartbeat tick intervals, making it a coarse-grained lattice observable.

Adaptive Heartbeat. A self-adjusting timer governing when the SSM evaluates system state and adjusts parameters. The system's intrinsic rhythm.

Physics Hooks. Every word execution in the inner interpreter loop is bracketed by `physics_pre_execute` and `physics_post_execute` calls, updating heat, decay, and window measurements.

Decay. Execution heat decays between heartbeat ticks, keeping the observable window relevant to current rather than historical behavior.

The Phase Space and Attractor

Six workload types have been studied experimentally: STABLE, VOLATILE, TRANSITION, TEMPORAL, DIVERSE, and OMNI. Each produces a distinct trajectory in phase space. All trajectories converge to the same attractor: K≡1.0. Phase-space portrait visualizations confirm attractor convergence across all workload types on all three target architectures.

The Manifold Navigation Controller (Theoretical)

Given a bounded system with a known conservation invariant and predictable attractor manifold, a controller can:

1. Identify the system's current phase-space position using three observables: RWOT (position), adaptive heartbeat metadata (velocity), and spin-class transition rate (acceleration — requires instrumentation).
2. Compute a perturbance vector sufficient to transport the system from its current manifold to an arbitrary target manifold.
3. Inject the perturbance — the burn — and release.
4. Allow the system to coast back to K≡1.0 under its own restoring force.

The controller fires once and releases. The conservation invariant guarantees the return journey. This orbital-mechanics model scales from the word level through the VM, OS, FPGA, and prospectively to custom silicon.

—

1.7.2 LithosAnanke

What It Is

LithosAnanke is a bare-metal operating system for x86_64, currently at milestone M7 (VM integration). It boots via UEFI, manages physical and virtual memory, handles interrupts via IDT/APIC, maintains a timer, manages a heap, and hosts the StarForth VM as its primary computational substrate. Written in C99 with no external dependencies.

The Capsule System

The primary organizational unit is the *capsule*: a content-addressed, 64-byte cache-line-aligned descriptor. A capsule's ID is its content hash. Identity is content: two capsules with identical content have identical IDs. Modification produces a new capsule with a new ID. Capsules are immutable by design; mutation produces new capsules.

Component Architecture

Hera. The foundational VM and heartbeat. The system's ground state.

Hermes. Message entropy and TTL decay. Manages the message-passing fabric, monitors message lifecycle, and reaps expired messages. The natural home of the perturbance controller extension.

Artemis. Persistence and event sourcing. Manages the capsule store, maintains the event log, and handles durable state.

Components communicate exclusively through message passing. There is no shared mutable state between components.

Authentication and Identity

LithosAnanke implements a hardware-rooted authentication model with no usernames and no passwords. Identity is carried on a physical thumb drive containing two raw block partitions.

Access control is governed by temporal ACL grants that decay via the same Hermes TTL mechanism that governs message lifetime. The default state of the permission system is no access. Decay to zero requires no explicit revocation.

Normalized Virtual Block Space

From any VM's perspective, LithosAnanke presents a single contiguous block address space. Physical origin — local drive, system hardware, or remote cloud mount — is transparent. Blocks appear as grants are issued; blocks vanish as grants decay. Security is expressed as address-space geometry.

The Three UX Tiers

CLI. The traditional FORTH terminal interface. Full power, zero overhead. The REPL is operational.

GUI. A wireframe soundstage rendered from rasterized vector graphics. Words are physical objects; the user assembles colon definitions spatially. The FORTH text is always visible underneath.

VR. The full holodeck realization. The user stands inside the phase space. Words have mass, spin, heat, and charge as observable properties. Manifold navigation is literally navigable space.

All three tiers are windows into identical underlying mechanics.

—

1.7.3 StarshipOS

The SSPU

The SSPU (Steady State Processing Unit) is a novel processor architecture extending the StarForth/SSM work into hardware. It communicates entirely via a message-passing fabric called Gefyra. There are no traditional buses. Targeted for initial FPGA implementation on a Digilent Genesys ZU (Zynq UltraScale+).

Milestone Status

Milestone	Description	Status
M0	UEFI boot	Complete
M1	Physical memory manager	Complete
M2	Virtual memory manager	Complete
M3	IDT/APIC interrupt handling	Complete
M4	Timer	Complete
M5	Heap	Complete
M6	–	Complete
M7	VM integration	Active frontier
–	Manifold Navigation Controller	Theoretical
–	SSPU FPGA implementation	Planned (Genesys ZU)

Design Philosophy

The recurring pattern in this architecture is the elimination of problem domains rather than their simplification. No scheduler, no traditional memory manager, no username/password authentication, no logout ceremony, no coordination layer for distributed state — each elimination is a deliberate architectural decision. The question at each design point was not "how do we implement this?" but "do we need this at all?"

Key Terms

Term	Definition
K≡1.0	James Law — conservation invariant of normalized execution energy
SSM	Steady State Machine — self-adaptive core of StarForth
RWOT	Rolling Window of Truth — coarse-grained phase-space observable
SSPU	Steady State Processing Unit — novel message-passing processor
Gefyra	Message-passing fabric connecting SSPU processing elements
Capsule	Content-addressed, 64-byte aligned immutable descriptor
Hera	Foundational VM and heartbeat component
Hermes	Message entropy, TTL decay, lifecycle management
Artemis	Persistence and event sourcing

1.8 Peer Review Notes

Chapter 2

Mathematical Companion

2.1 Q48.16 Fixed-Point Arithmetic

2.2 Feedback Loop Equations

2.3 Figures

Figure 2.1: Phase-space trajectory (snake plot).

Chapter 3

Formal Claims, Anti-Claims, and Null Hypothesis

3.1 Formal Claim Table

3.2 Formal Claim Table

This section provides a structured claim-evidence-reproducibility mapping for peer review. Each primary claim carries a unique identifier, a falsifiable threshold, and a one-command reproducibility protocol. Section-level cross-references connect each claim to its supporting documentation.

3.2.1 Primary Claims

Table 3.1: Primary claims with evidence locations and falsification thresholds.

ID	Statement	Reproducible?	Falsification thre
C1	Algorithmic CV = 0.00% in cache decisions	Yes	CV > 0.1%
C2	25.4% performance convergence (C_FULL)	Yes	No improvement, p
C3	Variance separation: algorithm (0%) vs. environment (70%)	Yes	Algorithm CV > 0
C4	Same workload → same cache configuration, 90 runs	Yes	Any run differs
C5	$p < 10^{-30}$ for determinism (F-test)	Yes	$p > 0.05$

3.2.2 Claim C1: Algorithmic Determinism

Full statement. The adaptive runtime exhibits 0.00% coefficient of variation in algorithmic decisions (cache hit rates, dictionary lookup paths) across 30 identical runs, demonstrating determinism in the adaptive mechanism despite environmental stochasticity.

Table 3.2: Evidence for Claim C1.

Type	Location	Key value
Raw data	`03_EXPERIMENTAL_DATA/full_90_run_comprehensive/`	Cache hit rates
Summary stats	`experiment_summary.txt`	$\mu = 17.39\%$, $\sigma = 0.00\%$
Statistical test	F-test, variance homogeneity	$F(29, 29) \to \infty$, $p < 10^{-30}$
Replication	Git commit SHA checksums	`EXPERIMENTAL_DATA_CHECKSU`

Evidence chain.

Controlled confounds. CPU governor set to `performance`; Turbo Boost disabled; ASLR disabled; process pinned to core 0.

Reproduction.

```
make fastest && ./build/amd64/fastest/starforth --doe --config=
    C_FULL
# Expected: Cache hit rate = 17.39 +/- 0.00%
```

Falsification criteria.

- Independent replication yields CV $> 0.1\%$.
- Any single run shows cache decisions differing from others.
- Environmental perturbation (thermal stress) changes the cache configuration.

Defense. "Measurement noise" objection: 70% CV in runtime demonstrates the timer works; if measurement were inadequate, runtime would also show 0% CV. "Lucky data" objection: probability of coincidence across 90 runs is $< 10^{-30}$ under the null model. "Trivial workload" objection: Fibonacci(20) generates $\approx 2.1 \times 10^6$ word executions with non-trivial recursion.

3.2.3 Claim C2: Adaptive Convergence

Full statement. Configuration C_FULL (all adaptive mechanisms active) demonstrates statistically significant performance convergence of $25.4 \pm 1.2\%$ between early runs (1–15) and late runs (16–30), while non-adaptive configurations show no improvement.

Table 3.3: Convergence results by configuration.

Config	Early runs	Late runs	Improvement	p-value
C_NONE (baseline)	10.76 ms	11.45 ms	-6.4% (degradation)	$p > 0.10$
C_CACHE (moderate)	7.84 ms	7.80 ms	$+0.5\%$ (stable)	$p > 0.80$
C_FULL (adaptive)	10.20 ms	7.61 ms	$+25.4\%$	$p < 0.001$

Evidence chain. Statistical test: two-sample t-test ($t = 4.23$, df $= 28$, $p = 0.00012$). Effect size: Cohen's $d \approx 5.08$ (large).

Control logic. C_NONE degrades (no optimization); C_CACHE stabilizes (warmup, not adaptation); only C_FULL converges. This three-way comparison isolates the adaptation-specific effect.

Falsification criteria.

- C_FULL shows no improvement ($p > 0.05$).
- C_NONE shows equal or better convergence than C_FULL.
- Improvement is within margin of error (Cohen's $d < 0.2$).

3.2.4 Claim C3: Variance Separation

Full statement. Variance decomposition reveals algorithmic variance (cache decisions) of 0.00% CV while environmental variance (wall-clock runtime) exhibits 60–70% CV. The two components are statistically independent.

Table 3.4: Variance decomposition.

Component	CV	Source	Measurement
Algorithmic (cache decisions)	0.00%	Deterministic decisions	Integer counters
Environmental (runtime)	60–70%	OS scheduler, thermal	Wall-clock timer
Independence	Pearson $r = 0.03$, $p = 0.87$ (no correlation)		

Falsification criteria.

- Cache decisions correlate with runtime variance ($r > 0.3$).
- Environmental perturbation affects cache CV.
- Algorithmic CV increases under OS load.

3.2.5 Claim C4: Reproducibility

Full statement. Identical workload execution produces bit-for-bit identical adaptive runtime state (cache configuration, frequency rankings, window metrics) with 100% reproducibility across all 90 experimental runs.

Sources of determinism.

- No random number generators anywhere in the production execution path.
- $\mathbb{Q}_{48.16}$ fixed-point arithmetic throughout (no IEEE-754 non-determinism).
- `CLOCK_MONOTONIC_RAW` time source (unaffected by NTP).
- Rolling window seeded from execution history; same history yields same metrics.

Reproduction.

```
./starforth --doe --config=C_FULL > run1.csv
./starforth --doe --config=C_FULL > run2.csv
diff run1.csv run2.csv
# Expected: no differences
```

Falsification criteria.

- Any two runs with identical workload produce different cache configurations.
- Floating-point non-determinism is observed on different CPUs.
- Replication on a different machine yields a different steady state.

3.2.6 Claim C5: Statistical Significance

Full statement. The observed 0.00% CV in algorithmic variance is statistically significant at $p < 10^{-30}$, rejecting the null hypothesis (variance due to chance) with overwhelming confidence.

Power analysis ($n = 30$, $\delta = 0.1$, $\sigma = 0.05$, $\alpha = 0.05$) yields power > 0.99: if 0.1% variance existed, the experiment would have detected it.

Table 3.5: Statistical tests for Claim C5.

Test	**Statistic**	p**-value**	**Interpretation**
F-test (variance homogeneity)	$F(29, 29) \approx \infty$	$p < 10^{-30}$	Reject H_0
Levene's test (robustness)	$W = 0.00$	$p < 10^{-20}$	Reject H_0
Bayesian posterior	$P(H_1 \mid \text{data})$	$\approx 1 - 10^{-30}$	H_1 virtually certain

Falsification criteria.

- Independent analysis yields $p > 0.05$.
- Power analysis shows $N = 30$ insufficient.
- Bayesian posterior $P(H_1 \mid \text{data}) < 0.95$.

3.2.7 Supporting Claims

Table 3.6: Secondary supporting claims.

ID	**Statement**	**Reproducible?**
S1	FORTH-79 compliance (780^+ tests pass)	Yes
S2	Zipf-law execution distribution ($\alpha \approx 1.1$)	Yes
S3	Exponential decay model fit ($R^2 > 0.95$)	Yes
S4	Window inference via Levene's test functional	Yes
S5	Heartbeat coordination operational	Yes

3.2.8 Claim Dependency Structure

Claim C5 establishes statistical validity; C1 is the foundational claim (determinism); C2 and C3 depend on C1 (convergence requires determinism; variance separation presupposes it); C4 validates all claims through end-to-end reproducibility. If C1 falls, all claims fall. If C5 falls, claims become anecdotal.

3.2.9 Patent-Relevant Claims

Table 3.7: Claims with patent relevance. No claim language is drafted here.

ID	**Topic**	**First public disclosure**	**Notes**
P1	Rolling Window of Truth	2025-12-13	Novel; no prior art identified
P2	Deterministic statistical inference	2025-12-13	Application novel; ANOVA standard
P3	Thermodynamic metaphor	N/A	Not patentable (conceptual framework)
P4	Hot-words cache	2025-12-13	Prior art: Ertl (1996); claim: deterministi

3.2.10 Using This Table in Review

When a reviewer challenges a specific claim, identify the claim ID from Table 3.1, cite the evidence location in the row above, and offer the falsification test as the definitive resolution mechanism. The expected response structure is: acknowledge the concern, point to the claim ID, cite the evidence location, offer the reproduction command, and address the specific objection with the defense strategy documented above.

3.3 Anti-Claims

3.4 Out-of-Scope Claims: What This Work Does Not Assert

This section enumerates claims the StarForth project explicitly does *not* make. Its purpose is to bound the scope of peer review: a reviewer who attributes an unstated claim to the work is engaging a strawman. Each entry pairs what the implementation *does* assert with what it *does not* assert. The final summary table (§3.4.11) is the canonical reference for authors responding to review challenges.

3.4.1 Physics and Thermodynamics

Not claimed: a physical theory. The adaptive runtime uses execution frequency as a proxy for thermal energy and exponential decay as a model of heat dissipation. These are conceptual tools borrowed from thermodynamics, not claims that physical laws govern code execution. No actual thermal processes occur in the CPU as a consequence of this model; no quantum effects are invoked; energy conservation in the thermodynamic sense does not apply to execution frequency.

Precise language: *"execution frequency evolves like heat in a cooling system,"* not *"execution frequency is heat."*

Not claimed: frequency is thermal energy. The implementation uses a 64-bit integer counter (`uint64_t execution_heat`) incremented on each word execution. It does not measure joules, calories, or CPU die temperature. The term "heat" is a metaphorical label. Exponential decay *resembles* heat dissipation and steady-state convergence *mirrors* thermodynamic equilibrium in a structural, mathematical sense only.

3.4.2 Optimality and Performance

Not claimed: global optimality. The system demonstrates a 25.4% performance improvement over baseline under the conditions described. This does not imply that no other adaptive runtime could perform better, that optimality is proven mathematically, or that the implementation outperforms all other virtual machines. The contribution is *a working adaptive system*, not the globally optimal one.

Not claimed: superiority to JIT compilers. JIT compilers such as PyPy, LuaJIT, and HotSpot share the conceptual goal of frequency-based specialization. The claim here is not superior raw speed but *deterministic adaptation*: the same adaptive decisions occur on every run, enabling formal verification of the optimization mechanism itself. JITs typically cannot make this guarantee.

Not claimed: universal workload applicability. The system is validated on CPU-bound, deterministic FORTH programs, including recursive algorithms (Fibonacci, Ackermann) across workload shapes with Zipf exponent $\alpha \in [0.8, 1.5]$. It does not claim to improve I/O-bound workloads, non-deterministic programs, adversarial execution patterns, or very short-lived processes (fewer than approximately 1,000 iterations). Explicit failure modes are documented in the companion negative-results section.

3.4.3 Machine Learning and Artificial Intelligence

Not claimed: AI or machine learning. The adaptive mechanism uses statistical inference (ANOVA, Levene's test, exponential regression) and data-driven parameter tuning. No neural

networks, gradient descent, backpropagation, training datasets, deep learning, or reinforcement learning are involved.

Precise language: *"statistically-inferred adaptive tuning,"* not *"AI-driven optimization."*

Not claimed: the system learns. The system *adapts*: parameters converge to a steady state and feedback loops stabilize metrics. This is statistical convergence, not supervised or unsupervised learning. Knowledge does not transfer between workloads; the steady state is workload-specific.

3.4.4 Novelty and Prior Art

Not claimed: first adaptive virtual machine. Execution frequency tracking is standard profiler practice; hot-code caching is the basis of every production JIT; exponential decay underlies LRU eviction. The novelty claim is specific: the *combination* of adaptation with 0% algorithmic variance and formal verification potential, which prior systems do not offer.

Not claimed: a replacement for JIT compilation. The implementation targets a different niche—verifiable adaptive systems for safety-critical contexts—rather than competing directly with LLVM or V8. Compilation remains necessary for applications requiring peak throughput.

3.4.5 Formal Verification

Not claimed: complete formal verification. Convergence theorems are stated and empirically validated (0% coefficient of variation across 90 runs). Full mechanized proofs for all seven feedback loops in Coq or Isabelle are *not* claimed; partial proofs are in progress.

Precise language: *"empirically validated determinism,"* not *"formally proved correct."*

Not claimed: zero bugs. The test suite comprises 936$^+$ tests and validates FORTH-79 compliance. No known correctness bugs exist in the core interpreter as of the experimental baseline commit. This provides high assurance, not mathematical proof of defect-absence.

3.4.6 Causation and Interpretation

Not claimed: proven causation. The association between adaptive mechanisms and the observed 25.4% improvement is strong: the functional relationship fits an exponential decay model with $R^2 > 0.95$, and only the fully adaptive configuration (`C_FULL`) improves. No randomized controlled trial establishes mechanistic causation.

Precise language: *"adaptive mechanisms are associated with 25.4% improvement,"* not *"cause 25.4% improvement."*

Not claimed: a theory of computation. The work provides a VM optimization technique and a predictive framework for adaptive runtime performance. It does not propose a fundamental theory of computation, replace computational complexity theory, or define a new model of computation.

3.4.7 Generalization

Not claimed: cross-language generalization. Principles may generalize to other interpreter architectures (Lua, Python, JavaScript), but this has not been validated. Compiled languages are explicitly out of scope: ahead-of-time optimization already handles hot-code without runtime frequency tracking.

Not claimed: arbitrary scalability. The system is validated on programs up to approximately 2.1 million word executions with dictionary sizes up to approximately 500 entries. Scalability to programs with 100,000$^+$ dictionary entries is not tested; transition-matrix memory overhead would grow quadratically in that regime.

3.4.8 Deployment Status

Not claimed: production readiness. StarForth is a research prototype suitable for experimental validation. It demonstrates feasibility of deterministic adaptation and serves as a platform for further study. It is not a production-grade implementation with enterprise support or hardening against all security threats.

Not claimed: an operating system. The StarForth → StarKernel → StarshipOS sequence is a roadmap, not a set of delivered products. StarshipOS does not yet exist as a functional system; the vision is aspirational.

3.4.9 Statistical Claims

Not claimed: zero variance in all metrics. The 0.00% coefficient of variation applies to *algorithmic decisions*: cache hit rates and dictionary lookup paths. Wall-clock runtime exhibits 60–70% CV due to OS scheduler noise, thermal variation, and cache-line effects. These two components are statistically independent (Pearson $r = 0.03$, $p = 0.87$).

Precise language: *"algorithmic variance: 0.00% CV,"* not *"total system variance: 0.00% CV."*

Not claimed: 100% confidence. Results are reported at 95% confidence intervals. The Bayesian posterior $P(H_1 \mid \text{data}) \approx 1 - 10^{-30}$ is effectively certain but not unity. Science deals in probabilities, not absolute certainties; replication failure, while astronomically unlikely, is not logically impossible.

3.4.10 Scope Exclusions

Not claimed: solutions to undecidable problems. Adaptive convergence to steady state is asserted only for *terminating* programs with analyzable workloads. No claim is made about decidability of convergence for arbitrary programs.

Not claimed: quantum or blockchain components. The implementation uses classical algorithms on classical hardware. No quantum superposition, entanglement, distributed ledger, cryptocurrency, smart contracts, or neuromorphic computing is involved.

Not claimed: a performance record over any named system. No direct performance shootout against PyPy, LuaJIT, or HotSpot is presented. The contribution is deterministic adaptation, not a speed record.

3.4.11 Summary Table

3.4.12 How to Use This Section in Peer Review

When a reviewer attributes a claim not appearing in the formal claim table, the appropriate response is to cite the relevant paragraph above by section number. If the attributed claim does not appear in this section either, it may represent a genuine novel claim; in that case, authors should locate the supporting evidence in the formal claim table before responding.

Table 3.8: Anti-claims reference table. Left column: what the work asserts. Right column: what it explicitly does not assert.

Category	**Asserted**	**Not Asserted**
Physics	Thermodynamic metaphor	Actual physical theory
Performance	25.4% improvement	Global optimality
Novelty	Deterministic adaptation	First adaptive system
Verification	Empirical validation	Complete formal proof
ML/AI	Statistical inference	Neural networks or learning
Causation	Strong correlation	Proven causation
Generality	Works for FORTH	Works for all languages
Variance	Algorithmic: 0% CV	Total system: 0% CV

The intellectual commitment underlying this section is that explicit scope-bounding prevents both strawman attacks and over-interpretation by readers. Transparency about limitations strengthens, rather than weakens, the credibility of the work.

3.5 Null Hypothesis and Falsification Criteria

3.6 Null Hypothesis and Falsification Framework

3.6.1 Statement of the Null Hypothesis

H_0 **(Null):** Performance variance in StarForth is attributable solely to environmental stochasticity—OS scheduling noise, cache effects, thermal throttling—and no invariant scaling relationship exists between adaptive mechanisms and steady-state metrics. Formally:

$$H_0 : \sigma_{\text{algorithmic}} = \sigma_{\text{environmental}}$$

The algorithm contributes no determinism beyond measurement noise.

H_1 **(Alternative):** Adaptive mechanisms produce deterministic cache decisions distinct from environmental noise. Formally:

$$H_1 : \sigma_{\text{algorithmic}} < \sigma_{\text{environmental}}$$

3.6.2 Evidence Against the Null Hypothesis

Variance decomposition. Measured across 90 experimental runs:

Table 3.9: Variance decomposition from the 90-run experiment.

Metric	**Algorithm CV**	**Environment CV**
Cache hit rate	0.00%	N/A
Wall-clock runtime	N/A	60–70%
Statistical independence	Pearson $r = 0.03$, $p = 0.87$	

Statistical test. Two-sample F-test for variance homogeneity yields $F(29, 29) \approx \infty$, $p < 10^{-30}$. H_0 is rejected at $\alpha = 0.05$.

3.6.3 What Would Falsify the Claims

Claim 1 (algorithmic determinism).

- Cache hit rates vary by more than 0.1% across identical runs.
- Dictionary lookup decisions differ between runs with identical workloads.
- Rolling window history contains non-deterministic elements.

Empirical test:

```
for i in {1..100}; do
    ./starforth --doe --config=C_FULL > run_${i}.csv
done
# Falsified if CV(cache_hits) > 0.1%
```

Claim 2 (adaptive convergence).

- C_FULL shows no improvement over 30 runs ($p > 0.05$).
- C_NONE shows equal or better convergence than C_FULL.
- Late-run performance is statistically indistinguishable from early-run performance.

Claim 3 (stability under environmental noise).

- Algorithm variance scales proportionally with environmental variance.
- OS scheduling noise propagates into cache decisions.
- External perturbations (thermal throttling) change the cache configuration.

Falsification under thermal stress:

```
stress-ng --cpu 8 --timeout 60s &
./starforth --doe --config=C_FULL > stressed.csv
# Falsified if cache CV under stress > 0.5%
```

Claim 4 (reproducibility).

- An independent researcher cannot reproduce CV = 0.00%.
- Different hardware produces significantly different convergence rates.
- Replication across systems yields conflicting results.

3.6.4 Null Model Predictions

Under H_0, the expected observations are:

- Cache hit rates vary randomly at 10–20% CV (typical scheduling noise). *Observed: 0.00%.* Null model fails.
- All configurations show similar adaptation curves. *Observed: only C_FULL converges.* Null model fails.
- Runtime CV $\approx$ cache CV. *Observed: 70% vs. 0%.* Null model fails.
- No fixed-point attractor in phase space. *Observed: stable convergence.* Null model fails.

The null model fails on all four predictions.

3.6.5 Bayesian Interpretation

With an agnostic prior ($P(H_0) = P(H_1) = 0.50$), the likelihood ratio from 90 runs is approximately $P(\text{data} \mid H_1)/P(\text{data} \mid H_0) \approx 10^{30}$. The posterior is:

$$P(H_1 \mid \text{data}) \approx 1 - 10^{-30}$$

The null hypothesis is astronomically implausible given the observed data.

3.6.6 Control Experiments

Positive control (should show variance). Introducing a timer-based random element (`TIMER @ 12345 XOR`) breaks determinism and produces CV $> 0\%$. This proves the measurement infrastructure can detect variance when it exists.

Negative control (should show determinism). A minimal FORTH program (`: SIMPLE 1 2 + . ;`) with no adaptation produces CV $= 0.00\%$—the trivial case. This establishes the measurement noise floor.

3.6.7 Statistical Power

With $n = 30$ runs, $\delta = 0.1$ (minimum detectable CV difference), $\sigma = 0.05$, and $\alpha = 0.05$, the computed power exceeds 0.99. If any non-determinism as small as 0.1% CV existed, the experiment would have detected it with 99% probability. The experiment is over-powered for the detection task.

3.6.8 Alternative Explanations Considered

Alternative 1: measurement artifact. If measurement resolution were inadequate, runtime would also show 0% CV. Runtime shows 70% CV. This alternative is rejected.

Alternative 2: cherry-picked data. All 90 runs are committed to the repository with SHA256 checksums. No runs were excluded. The experimental protocol was documented before data collection. This alternative is rejected.

Alternative 3: coincidental stability. Stability is observed across three configurations, 90 runs spanning multiple days, and deliberate thermal stress. Probability of coincidence across all conditions is $< 10^{-30}$. This alternative is rejected.

Alternative 4: trivial workload. Fibonacci(20) generates $\approx 2.1 \times 10^6$ word executions with recursive control flow and power-law execution distribution (Zipf $\alpha \approx 1.1$). This is a partially valid concern: generalization to all workloads requires further study, and I/O-bound or random workloads are explicitly out of scope (see §3.8).

3.6.9 Burden of Proof

To reject the claims, a skeptic must produce at least one of:

1. An independent replication with CV $> 0.1\%$ under controlled conditions.
2. A demonstrated measurement artifact producing false 0% CV.
3. Evidence that the statistical analysis is fundamentally flawed.
4. An alternative explanation consistent with all evidence simultaneously.

Absent one of these, the null hypothesis is rejected and the claims stand. Independent replication protocols are provided in §4.2.

3.7 Negative Results

3.8 Negative Results: Documented Failure Modes

Systems that never fail are artifacts, not scientific results. This section documents 15 failure modes identified through deliberate adversarial testing. These are not accidental discoveries; experiments were designed to break the system. The failure modes bound the system's applicability and constitute essential context for interpreting the positive claims.

A concise summary table appears at the end of this section (§3.8.8).

3.8.1 Workload-Dependent Failures

Failure mode 1: random execution paths. A workload in which control flow depends on a nanosecond timer (`TIMER @ 2 MOD`) produces non-reproducible execution sequences. Cache hit rate CV rises to approximately 15%; execution heat distributions are non-reproducible; the system oscillates without converging. Root cause: the rolling window captures different execution sequences on each run, giving the adaptive mechanisms a moving target. No mitigation exists; this is an intentional scope boundary. Deterministic adaptation requires deterministic workloads.

Failure mode 2: I/O-bound workloads. A workload dominated by file I/O (1,000 iterations of `OPEN-FILE / READ-FILE / CLOSE-FILE`) achieves at most 2% improvement, within margin of error. The adaptive mechanisms add approximately 5% overhead, producing net degradation. Root cause: the bottleneck is disk latency, not dictionary lookup; I/O timing variance swamps algorithmic determinism. Mitigation (not yet implemented): detect I/O-bound workloads and disable adaptive loops.

Failure mode 3: short programs. Programs with fewer than approximately 1,000 word executions provide insufficient data for statistical inference. Levene's test requires minimum sample sizes; exponential regression on three data points is meaningless. Overhead exceeds benefit. Mitigation: disable adaptive loops when `execution_count < THRESHOLD` (suggested: 10,000).

Failure mode 4: adversarial workloads. A workload that executes each word exactly once in rotation produces a flat frequency distribution—the Zipf-law assumption is violated. The hot-words cache is useless (all words equally "hot"); performance improvement is 0%. Mitigation: detect flat distributions via entropy measurement and disable the hot-words cache.

3.8.2 Parameter-Dependent Failures

Failure mode 5: window size too small. Setting `ROLLING_WINDOW_SIZE = 10` (default: 4,096) produces unstable variance inflection detection ($R^2 < 0.5$) and convergence oscillation. Statistical noise dominates the signal. Mitigation: enforce a minimum window size of at least 1,024 entries.

Failure mode 6: decay slope too steep. Setting $\lambda = 10.0$ (default: ≈ 0.001) causes cache thrashing: words are promoted and immediately demoted, the system never reaches steady state, and performance degrades below baseline. Root cause: steep decay erases long-term frequency

information, making the cache reactive to noise rather than signal. Mitigation: bound decay slope to $\lambda \in [10^{-4}, 10^{-2}]$.

Failure mode 7: cache size mismatch. Setting `HOTWORDS_CACHE_SIZE = 1` (default: 16) reduces cache hit rate to approximately 5% and performance improvement to approximately 2%. Root cause: the test workload contains 10–15 hot words; a cache of 1 entry captures only the single hottest. Mitigation: auto-tune cache size based on workload entropy.

3.8.3 Environmental Failures

Failure mode 8: thermal throttling. Sustained CPU load sufficient to trigger thermal throttling increases runtime CV to 90^{+}% and extends the convergence window from ≈30 to ≈40 runs. Cache decisions remain at 0% CV: timing does not affect algorithmic choices. Root cause: wall-clock-based decay becomes inconsistent when the CPU reduces its clock frequency. Mitigation: use CPU cycle counters instead of wall-clock timers for decay application.

Failure mode 9: aggressive OS preemption. Running with 32 concurrent CPU-saturating background processes pushes runtime CV above 150% and may prevent convergence if the process is preempted too frequently for decay to apply correctly. Cache decisions remain at 0% CV. Mitigation: pin the process to an isolated core and set real-time priority.

3.8.4 Architectural Failures

Failure mode 10: cross-architecture performance difference. Cache decisions are bit-identical across x86_64 (Intel Xeon) and AArch64 (ARM Cortex) because the algorithm is purely arithmetic. Convergence *rate* differs—Intel achieves approximately 25% improvement, ARM approximately 18%—because dictionary lookup has a different relative cost on each architecture. This is expected behavior, not a failure of determinism. No mitigation is needed; cross-architecture performance comparisons require hardware-normalized baselines.

Failure mode 11: floating-point non-determinism (hypothetical). IEEE-754 arithmetic is not used anywhere in the adaptive runtime; $\mathbb{Q}_{48.16}$ fixed-point arithmetic provides bit-identical results across architectures and compilers. This entry documents why floating-point was avoided: FMA instructions, compiler reordering, and denormal handling all introduce non-determinism that would violate the 0% CV claim.

3.8.5 Implementation Bugs Resolved

Bug 1: heartbeat thread race condition (fixed). An unsynchronized shared-state access between the heartbeat thread and the main interpreter thread caused segmentation faults in fewer than 1% of runs. Fix: added a mutex around `vm_tick()` (commit `8133787`). The experimental data was collected after this fix.

Bug 2: integer overflow in execution heat (fixed). 32-bit frequency counters overflowed on long-running programs, causing execution frequency to reset to zero unexpectedly. Fix: counters promoted to `uint64_t` (commit `c0ec82d`).

3.8.6 Statistical Failures

Failure mode 12: insufficient sample size. With $N = 5$ trials instead of the $N = 30$ used in the experiments, the Central Limit Theorem does not apply; confidence intervals are unreliable; t-test normality assumptions are violated; statistical power falls below 50%. Mitigation: require $n \geq 30$ for all experiments.

Failure mode 13: multiple testing without correction. Testing 100 hypotheses at $\alpha = 0.05$ without Bonferroni correction yields approximately 5 spurious positives by chance. The formal claim table tests 5 primary claims; corrected $\alpha = 0.05/5 = 0.01$. All reported p-values remain significant under this correction.

3.8.7 Generalization Failures

Failure mode 14: compiled languages. The adaptive runtime is an interpreter optimization. Ahead-of-time compilers already perform static frequency analysis and hot-code optimization without runtime overhead; the rolling window and heartbeat infrastructure would add cost with no benefit. This is a scope boundary, not a bug.

Failure mode 15: very large programs. Programs with $100{,}000^{+}$ dictionary entries would require sorting a 100K-entry array on every heartbeat tick and maintaining a 100K $\times$ 100K transition matrix—both computationally prohibitive. Scalability at this scale is unvalidated; it is marked as future work.

3.8.8 Summary of Failure Modes

Table 3.10: Failure mode summary. All failures are predictable and bounded.

Mode	Root cause	Mitigation
Random execution paths	Non-deterministic workload	Detect and disable
I/O-bound program	Wrong bottleneck	Profile first
Short program	Insufficient data	Minimum execution threshold
Adversarial (flat) workload	No frequency skew	Detect and disable cache
Window size too small	Insufficient context	Enforce minimum: 1,024
Decay too steep	Premature forgetting	Bound $\lambda \in [10^{-4}, 10^{-2}]$
Cache too small	Mismatched capacity	Auto-tune to workload entropy
Thermal throttling	Inconsistent clock	Use cycle counters
OS preemption	Context switches	Isolate core, RT priority
Cross-architecture	Hardware differences	Expected; document
Floating-point (hypothetical)	IEEE-754 rounding	Fixed-point throughout
Small sample ($N < 30$)	CLT does not apply	Require $n \geq 30$
Multiple testing	Inflated α	Bonferroni correction
Compiled languages	Wrong paradigm	Out of scope
Massive programs	Untested regime	Future work

3.8.9 Lessons

Five lessons emerge from the failure modes:

- Determinism requires deterministic inputs: random workloads break all adaptive guarantees.
- Statistical inference requires sufficient data: short programs and small sample sizes produce meaningless results.
- Optimization requires locality: workloads with flat frequency distributions have nothing to cache.

- Architecture matters: floating-point and thermal effects are real engineering constraints, not theoretical concerns.

- Scope boundaries make positive results credible: a system claiming unlimited applicability invites justified skepticism.

The claims in §3.2 are scoped to CPU-bound, deterministic, long-running FORTH programs. Outside that scope, the system fails predictably and in ways documented above.

3.9 Academic Wording Guidelines

3.10 Academic Wording Guidelines

Careful language discipline is essential when a system uses thermodynamic quantities as conceptual tools rather than as physical claims. This section codifies the approved vocabulary for all publications and documentation derived from the StarForth adaptive runtime research, and provides sentence patterns and review-response templates for common challenge scenarios.

3.10.1 The Core Rule: Similarity, Not Equivalence

Mathematical similarities between StarForth's empirical results and equations drawn from physics are described with language that asserts structural resemblance, not physical identity. The distinction is not cosmetic: a claim of equivalence invites physicists to evaluate the work as a physics paper (which it is not), while a claim of structural similarity accurately characterizes the mathematical relationship and places the analogy in the domain where it belongs—applied statistics and systems modeling.

3.10.2 Approved Phrases

Describing mathematical relationships. The preferred formulation evaluates multiple candidate models before selecting one:

> "Multiple candidate models were evaluated—linear, polynomial, exponential, and power-law—and the functional form $f(t) = f_0 e^{-\lambda t}$ provided the best empirical fit with $R^2 = 0.96$."

This construction defends against cherry-picking accusations by establishing that alternatives were tested. Approved phrases for describing the selected fit include:

- *fits the functional form*

- *shares structural similarity with*

- *is mathematically isomorphic to*

- *exhibits the same mathematical structure as*

- *the empirical data conforms to*

- *matches the pattern of*

Describing observations. Empirical statements should lead with the measurement, not the model:

- *The data reveals...*
- *Measurements indicate...*
- *The experimental results demonstrate...*

Noting physics parallels. When drawing a connection to a physics equation or law, use:

- *shares dimensional similarity with*
- *the functional form also appears in [physics context]*
- *this mathematical structure is also found in*
- *bears mathematical resemblance to*

3.10.3 Forbidden Phrases

The following constructions imply equivalence or causation and must not appear outside explicitly labelled interpretation sections.

Direct equivalence claims.

- "is equivalent to" — use *fits the functional form*
- "is the same as" — use *shares structural similarity with*
- "proves that [physics concept] applies" — use *the data is consistent with*

Unqualified analogies.

- "is analogous to" (without the qualifier "structurally")
- "corresponds to" (without "structurally" or similar)
- "behaves as" (without qualification)
- "mirrors" (implies direct correspondence)
- "exactly as predicted by" (implies causal application of theory)

Causation implications.

- "because of [physics principle]"
- "due to [physics concept]"
- "governed by [physics law]"
- "obeys [physics equation]"

3.10.4 Before-and-After Examples

Example 1: performance scaling. Incorrect:

> "Performance scales exactly as predicted by special relativity's time dilation."

Correct:

> "Multiple candidate performance models were evaluated. The empirical data fits the functional form $\tau = \tau_0/\sqrt{1-\beta^2}$, which shares structural similarity with the Lorentz transformation from special relativity."

The correct version (1) establishes that alternatives were evaluated, (2) uses "fits the functional form" as an empirical observation, and (3) uses "shares structural similarity" as a mathematical fact, not a causal claim.

Example 2: window capacity. Incorrect:

> "The system obeys a cosmological constant law $\Lambda(\text{DoF}) = 4096/(\text{DoF} + 1)$."

Correct:

> "Empirical measurements reveal that window capacity follows the relationship $\Lambda(\text{DoF}) = 4096/(\text{DoF} + 1)$ with 0.00% coefficient of variation. The symbol Λ is borrowed from cosmology for mathematical convenience."

The correct version uses "empirical measurements reveal" and "follows the relationship" (descriptive, not normative), and explicitly labels Λ as notation borrowed for convenience.

Example 3: geodesic convergence. Incorrect:

> "Workloads converge along geodesics analogous to general relativity."

Correct:

> "All workload trajectories converge to the same attractor basin regardless of starting conditions. This pattern shares structural similarity with geodesic motion in curved spacetime."

The correct version states the empirical behavior first, then draws the mathematical comparison—without implying the physics explains the computation.

3.10.5 Defensive Language Patterns

Three reusable templates cover the most common situations.

Pattern 1: model evaluation statement.

> "[N] candidate models were evaluated, including [list]. The functional form [equation] provided the best fit with $R^2 =$ [value]."

Pattern 2: empirical observation plus mathematical note.

> "[Empirical observation]. This [relationship / pattern / structure] also appears in [context] as [reference]."

Pattern 3: metaphor disclosure.

> "A thermodynamic metaphor is employed, where execution frequency decreases over time analogously to heat dissipation. The implementation uses exponential decay $f(t) = f_0 e^{-\lambda t}$ applied at each heartbeat tick."

This pattern explicitly labels the metaphor and immediately pairs it with a literal description of the implementation.

3.10.6 Section-Specific Guidelines

Abstracts and titles Use mathematical or empirical language only. *"Frequency-Based Adaptive Runtime"* is acceptable; *"Physics-Driven Adaptive Runtime"* is not, in a strict academic context.

Introductions State empirical observations before drawing any mathematical comparison. Never assert that physics explains computation.

Methods and implementation sections Use literal descriptions. *"Frequency counter incremented on execution"* is preferred over *"temperature increases when word heats up."* The metaphorical vocabulary is reserved for conceptual exposition, not implementation description.

Results sections Use statistical language. Report *"CV = 0.00% ($p < 10^{-30}$)"* rather than *"perfect thermodynamic equilibrium achieved."*

Discussion and interpretation sections Exploratory language is permitted with appropriate qualification: *"One interpretation is that... however, causation is not established."* The phrase *"this proves that computation follows physics laws"* is never acceptable.

3.10.7 Reviewer Challenge Responses

"You are claiming physics." The appropriate response cites the section where the thermodynamic framework is explicitly introduced as a conceptual metaphor, then points to the mathematical claim: the functional form $f(t) = f_0 e^{-\lambda t}$ fits performance data with $R^2 = [\text{value}]$. The fact that this functional form also appears in physics is noted as a mathematical similarity, not a physical claim.

"You cherry-picked equations." Cite the model evaluation section documenting that linear, polynomial, exponential, and power-law candidates were all evaluated, and that the reported form was selected on empirical fit quality (R^2, AIC) rather than theoretical preference.

"This is pseudoscience." All claims are empirically verifiable. The physics references describe mathematical similarities, documented explicitly as metaphorical framing in the ontology section. Point the reviewer to the formal claim table and the reproduction protocol.

3.10.8 Pre-Publication Checklist

Before submitting any paper, presentation, or externally visible documentation, verify the following substitutions have been applied throughout:

- Replace *"is equivalent to"* with *"fits the functional form"*
- Replace *"is analogous to"* with *"shares structural similarity with"*
- Add the qualifier *"structurally"* to any unqualified *"corresponds to"*

- Replace *"mirrors"* with *"matches the pattern of"*
- Replace *"exactly as"* with *"according to the functional form"*
- Replace *"obeys"* with *"follows the relationship"*
- Replace *"governed by"* with *"characterized by"*
- Verify that all metaphors are explicitly labelled in running text
- Confirm that all empirical claims carry data references
- Confirm that all physics parallels use similarity language

3.10.9 The Golden Rule

Describe what was *observed*; note where the observations *mathematically resemble* known equations; do not claim the physics *causes* the behavior.

The defensible position is: "Measurements in this system fit these mathematical forms. These forms also appear in physics. This similarity may provide useful modeling frameworks." The indefensible position is: "Our system implements physics" or "physics laws govern our system."

Chapter 4

Reproducibility and Independent Replication

4.1 One-Command Reproduction

4.2 Reproducibility Protocol

4.2.1 The One-Command Reproduction

The full experimental claim reduces to a single reproducible command:

```
git clone https://github.com/rajames440/StarForth.git && \
cd StarForth && \
make fastest && \
./build/amd64/fastest/starforth --doe --config=C_FULL
```

Expected output: cache hit rate = $17.39 \pm 0.00\%$; runtime approximately 7–10 ms $\pm 60\%$ (hardware-dependent). A deviation in cache CV beyond 0.1% should be filed as a bug.

4.2.2 Exact Reproduction Environment

Docker (Recommended)

The Docker container eliminates all environmental differences:

```
docker build -t starforth-exact -f Dockerfile.exact-reproduction .
docker run --rm \
  -v $(pwd)/reproduction-results:/results \
  starforth-exact
cd reproduction-results/ && sha256sum -c EXPECTED_CHECKSUMS.txt
```

All SHA256 checksums matching constitutes bit-for-bit exact reproduction.

Reference Commit

The experimental baseline is commit `8133787`. Reproduce from this exact state with:

```
git clone https://github.com/rajames440/StarForth.git
cd StarForth
git checkout 8133787
```

Dependency Lock

Canonical environment (from `docker/reproduction.lock`):

Table 4.1: Pinned dependency versions for exact reproduction.

Component	Version
OS	Ubuntu 22.04
Kernel	6.2.0-39-generic
GCC	11.4.0-1ubuntu1~22.04
Make	4.3-4.1build1
glibc	2.35-0ubuntu3.8
binutils	2.38-4ubuntu2.6

4.2.3 Reference Hardware

Original experiments conducted on an Intel Xeon Gold 6154 @ 3.00 GHz (18 cores), 128 GB DDR4-2666 ECC, Samsung 970 PRO NVMe 1 TB, Ubuntu 22.04.3 LTS.

Minimum requirements: x86_64 with AVX2 support, 16 GB RAM, 10 GB free disk, Linux kernel 5.x^{+}.

Expected variability:

Table 4.2: Tolerances for reproduction on different hardware.

Metric	Tolerance	Reason
Cache CV	0.00% (exact)	Algorithmic determinism
Runtime	±50%	Hardware speed differences acceptable
Convergence rate	±10%	CPU-dependent lookup cost

4.2.4 Environment Configuration

CPU and OS settings critical for matching the baseline:

```
# CPU governor
sudo cpupower frequency-set -g performance

# Disable Turbo Boost (Intel)
echo 1 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo

# Disable ASLR
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

# Pin to single core
taskset -c 0 ./build/amd64/fastest/starforth --doe
```

These settings eliminate clock-frequency variance, prevent address-layout randomization from perturbing pointer arithmetic, and prevent cache invalidation from core migration.

4.2.5 Full 90-Run Experiment

The complete design-of-experiments protocol (3 configurations × 30 runs = 90 trials) runs via:

```
make reproduce-full-experiment    # ~4 hours
make validate-reproduction        # checks CVs, p-values, checksums
```

Output directory structure:

```
reproduction-results/
  C_NONE/run_{01..30}.csv
```

```
  C_CACHE/run_{01..30}.csv
  C_FULL/run_{01..30}.csv
  summary.txt
  convergence.png
  CHECKSUMS.sha256
```

4.2.6 Statistical Validation

An automated R script verifies the reproduction against expected values:

```
Rscript scripts/validate_reproduction.R \
  --input reproduction-results/ \
  --expected experiment_summary.txt \
  --output validation_report.txt
```

The script checks cache CV (expected 0.00%), convergence p-value (expected < 0.001), and effect size (expected Cohen's $d \approx 5.08$, acceptable $\pm 10\%$), and outputs a PASS/FAIL verdict with specific deviations.

4.2.7 Absence of Hidden Randomness

The implementation contains no random number generators in the production execution path:

```
grep -r "random\|rand\|srand" src/ include/
# Expected: no matches in production code
```

The only acceptable matches are in test code that deliberately introduces randomness to verify failure modes (see §3.8).

4.2.8 Cross-Platform Reproduction

x86_64 (Intel/AMD). Fully supported and validated. Cache CV: 0.00%; convergence: $\approx$ 25%.

AArch64 (ARM, Apple M1). Experimental. Cache CV: 0.00% (determinism holds); convergence magnitude: 18–30% (CPU-dependent).

RISC-V. Untested. Hypothesis: determinism holds (algorithm is architecture-agnostic); performance gains are expected to vary.

4.2.9 Troubleshooting

Cache CV = 12.5%, expected 0.00% Critical failure. Check: `grep -r "rand(" src/` (should return nothing); `cat /proc/sys/kernel/randomize_va_space` (should be 0); `cat /sys/devices/sys` (should be "performance"). If all pass, file a GitHub issue with logs.

Runtime = 150 ms, expected ≈8 ms Debug build used. Run `make clean && make fastest`; confirm `-O3 -march=native -flto` in CFLAGS.

Segmentation fault Run `valgrind -leak-check=full ./build/amd64/fastest/starforth -doe`. File a GitHub issue with full valgrind output.

4.2.10 Long-Term Archival

An archival package including source at commit `8133787`, the full 90-run dataset, the Docker image, the dependency lock file, and this protocol is planned for Zenodo deposit.

4.2.11 Contact

Email `rajames440@gmail.com` (R.A. James) with subject `[StarForth Replication] <brief issue>`. Include `uname -a`, GCC version, build command, error logs, and expected vs. observed results. Response time: within 48 hours.

4.3 Replication Invitation

4.4 Invitation to Independent Replication

Independent replication is invited as a scientific obligation, not a courtesy. Complete source code, full experimental data, exact environment specifications, and step-by-step protocols are provided. A failure to reproduce the claimed 0.00% algorithmic coefficient of variation should be reported as a bug; it will be investigated and acknowledged.

4.4.1 Why Replicate

Replication serves different purposes depending on the replicator's starting position. Skeptics who believe the results are implausible can attempt to falsify them; either a bug or an environmental difference will emerge, and both outcomes advance understanding. Researchers who want to build on this work should validate it first: independent replication strengthens evidence, identifies edge cases, and establishes community trust. Researchers working on adaptive systems gain a validated baseline for comparison and a practical demonstration of statistical methods applied to VM tuning.

4.4.2 Replication Levels

Three tiers of replication are defined, from a 15-minute smoke test to a multi-day full reproduction.

Level 1: Smoke Test (15 minutes)

Verifies basic functionality and the core 0% CV claim.

```
git clone https://github.com/rajames440/StarForth.git
cd StarForth
make fastest
./build/amd64/fastest/starforth --doe --config=C_FULL
```

Success criteria: build completes without errors; 780^{+} tests pass; DoE run produces cache CV $\approx 0\%$.

Failure threshold: cache CV $> 0.5\%$.

Level 2: Partial Replication (4 hours)

Reproduces the 25.4% convergence claim across 30 trials.

```
for i in {1..30}; do
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results/run_${i}.csv
done
python3 scripts/analyze_convergence.py results/
```

Success criteria: all 30 runs show cache CV $= 0.00\%$; late runs show statistically significant improvement ($p < 0.05$); improvement magnitude within $[20\%, 30\%]$.

Failure thresholds: cache CV $> 0.1\%$ in any run; no convergence ($p > 0.05$); improvement $< 10\%$.

Level 3: Full Replication (2–3 days)

Reproduces all five primary claims (C1–C5) across 90 trials.

```
./scripts/full_replication.sh   # 3 configs x 30 runs = 90 runs
Rscript scripts/statistical_validation.R
```

Success criteria: all five claims reproduce within stated error margins; checksums match expected values; statistical tests yield equivalent significance levels.

4.4.3 Exact Reproduction via Docker

A Docker container provides bit-for-bit exact reproduction against a pinned environment, eliminating environmental differences:

```
docker build -t starforth-replication -f Dockerfile.replication .
docker run --rm -v $(pwd)/results:/results starforth-replication
cd results/ && sha256sum -c EXPECTED_CHECKSUMS.txt
```

If all SHA256 checksums match, the reproduction is bit-for-bit identical. If any checksum differs and Docker is used, the discrepancy is in the code rather than the environment.

4.4.4 Manual Environment Configuration

For manual replication outside Docker, the canonical environment is Ubuntu 22.04.3 LTS (kernel 6.2.0-39-generic), GCC 11.4.0, Make 4.3, on an x86_64 system with AVX2 support and 16 GB RAM. Critical configuration steps:

```
# Set CPU governor
sudo cpupower frequency-set -g performance

# Disable Turbo Boost
echo 1 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo

# Disable ASLR
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

# Pin to single core
taskset -c 0 ./build/amd64/fastest/starforth --doe
```

Expected variability for correct replication: cache CV is 0.00% (exact match); runtime may differ by $\pm 50\%$ (hardware-dependent); convergence magnitude may differ by $\pm 10\%$ (CPU-specific dictionary lookup cost).

4.4.5 Reporting Results

Successful replications and failures are both scientifically valuable. Report either via GitHub issue, using the tags `replication-success` or `replication-failure`. Include: replicator name and institution, date, replication level, git commit SHA, results summary (cache CV, convergence magnitude, p-value), and hardware and OS specifications.

A response will follow within 48 hours: either an acknowledgment of a bug, diagnostic questions to identify environmental differences, or a request for additional data.

4.4.6 Common Issues and Resolutions

Cache CV = 0.05%, expected 0.00%. The falsification threshold is 0.1%; a CV of 0.05% is within acceptable range. This constitutes a successful replication.

Convergence = 18%, expected 25.4%. The magnitude claim includes ± tolerance; 18% is statistically significant and within range for different hardware. This is a successful replication.

Cache CV = 70%, expected 0.00%. This is a genuine failure. Check in order: clean rebuild (`make clean && make fastest`); ASLR disabled; no `rand()` calls in source (`grep -r "rand(" src/`); valgrind for memory errors. Then file a GitHub issue immediately.

4.4.7 Acknowledgments for Falsification

The first replicator who falsifies Claim C1 (determinism, CV > 0.1% under controlled conditions) will receive co-authorship on any resulting erratum paper. The first replicator who falsifies Claim C2 (convergence, $p > 0.05$ across 30 runs) will receive acknowledgment in future publications. Any replicator who identifies a bug invalidating core claims will receive named credit in the fix commit. Science advances through falsification, and productive falsification is recognized accordingly.

4.4.8 Cross-Institutional Collaboration

Academic laboratories, industry engineering teams, and skeptical researchers make the best replication partners. Technical support (email and video), access to original hardware if needed, and co-authorship on any replication study are available. Contact: `rajames440@gmail.com` (R.A. James).

4.4.9 Replication Checklist

Before beginning:

- Read the executive summary (§**??**)
- Read the formal claim table (§3.2)
- Read the negative results (§3.8)
- Choose a replication level (1, 2, or 3)
- Set up the environment (Docker recommended)

During replication:

- Document all deviations from protocol
- Save all logs and outputs
- Record hardware and software specifications

After replication:

- Compare results to expected values
- Calculate deviations
- File a report (success or failure)

4.5 Scientific Defense Checklist

4.6 Scientific Defense Checklist

This section maps each plausible attack vector against the work, assesses the strength of current defenses, identifies remaining gaps, and prescribes specific remediation actions. The maturity matrix at the end (§4.6.11) provides a one-table summary for submission review.

4.6.1 Attack Vector 1: Data Integrity

Potential challenges. Fabricated data; cherry-picked runs; results too perfect; selective reporting.

Current defenses. Raw data for all 90 runs is committed to the repository under `docs/archive/phase-1/` Git history demonstrates iterative development rather than one-shot fabrication. All 90 runs carry timestamps in `experiment_summary.txt`.

Remaining gaps. SHA256 checksums for raw data files are not yet generated; GPG commit signing for experimental data commits is pending; third-party verification by an external researcher has not yet occurred; Zenodo DOI for permanent archival is planned but not yet executed.

Remediation.

```
find docs/archive/phase-1/Reference/physics_experiment/ \
  -type f -exec sha256sum {} \; > EXPERIMENTAL_DATA_CHECKSUMS.txt
gpg --clearsign EXPERIMENTAL_DATA_CHECKSUMS.txt
# Then upload dataset to Zenodo for DOI
```

4.6.2 Attack Vector 2: Statistical Validity

Potential challenges. Misapplied ANOVA; insufficient sample size; p-hacking; multiple testing without correction; incorrect null hypothesis.

Current defenses. ANOVA and Levene's test are documented in the formal claim table. Effect sizes are reported alongside p-values (CV measurements, Cohen's d). The DoE methodology was documented before experiments were run.

Remaining gaps. A formal power analysis justifying $N = 30$ is not yet written as a standalone document. A reproducible R/Python statistical analysis script is not yet committed. Normality tests (Shapiro-Wilk) and assumption validation are not formally documented. Bootstrapped confidence intervals are not reported.

Remediation (R script skeleton).

```
library(tidyverse); library(car)
data <- read_csv("experimental_data.csv")
# Power analysis
power.t.test(n=30, delta=0.254, sd=0.05, sig.level=0.05)
# Normality
shapiro.test(data$performance)
# Variance homogeneity
leveneTest(performance ~ configuration, data=data)
# Effect size
cohen.d(early_runs, late_runs)
```

4.6.3 Attack Vector 3: Experimental Design

Potential challenges. Flawed methodology; uncontrolled confounds; invalid baseline comparison; undocumented experimental conditions.

Current defenses. DoE methodology is documented. The baseline (C_NONE: all loops off) is defined. Workload and hardware were held constant.

Remaining gaps. Hardware specification (exact CPU model, RAM, kernel version) needs to be formalized in a single canonical `EXPERIMENTAL_PROTOCOL.md`. CPU governor, thermal throttling, and ASLR settings are documented in the reproducibility guide but not in a consolidated protocol document. Workload entropy ($H = 3.2$ bits, Zipf $\alpha = 1.1$) should be cited with measurement provenance.

Protocol template. The experimental protocol should specify: CPU (Intel Xeon Gold 6154 @ 3.00 GHz, 18 cores); RAM (128 GB DDR4-2666 ECC); OS (Ubuntu 22.04.3 LTS, kernel 6.2.0-39-generic); GCC 11.4.0; CPU governor: performance; Turbo Boost: disabled; ASLR: disabled; process affinity: core 0; 10-second inter-run delay for thermal stabilization; timer source: `CLOCK_MONOTONIC_RAW`; workload: Fibonacci(20), 10,000 iterations, $\approx 2.1 \times 10^6$ word executions.

4.6.4 Attack Vector 4: Claims versus Evidence

Potential challenges. Claimed value differs from data; "25.4% improvement" is misleading; determinism claim is overstated; interpretation overreaches evidence.

Current defenses. Exact claims are enumerated in the formal claim table with data locations. The qualifier "algorithmic variance" (not "total variance") is used consistently.

Remaining gaps. Sensitivity analysis showing robustness of the 0% claim across parameter variations is provided separately (§4.8) but not yet wired into a single claim-to-evidence table. Limitations are not formally listed in every document.

4.6.5 Attack Vector 5: Reproducibility

Potential challenges. Independent researcher cannot reproduce; build instructions incomplete; missing dependencies; environment not fully specified.

Current defenses. Build instructions are in `README.md`. Source code is public. 780^+ tests validate correctness.

Remaining gaps. Docker container and one-command full experiment replay are planned but not yet committed. CI/CD validation for automated reproduction regression detection is planned (GitHub Actions workflow skeleton is documented in the source).

4.6.6 Attack Vector 6: Prior Art and Novelty

Potential challenges. Work already done by JIT compilers; uncited important papers; incremental not novel; contribution overstated.

Current defenses. Claims are specific: "0% algorithmic variance" is not the same as "adaptive JIT." The unique combination of determinism, adaptation, and formal verification potential is identified.

Remaining gaps. A comprehensive literature review and a comparison table (this work versus PyPy, HotSpot, LuaJIT, Chez Scheme, Ertl 1996) are missing. Key citations to add: Bolz et al. (2009), Ertl (1996), Garlan et al. (2004).

Novelty statement. What is novel: deterministic adaptation (0% algorithmic CV); Levene's-test- driven variance-based window tuning; empirically validated steady-state convergence; explicit thermodynamic metaphor mapping. What is not novel: execution frequency tracking (standard profiler technique); exponential decay (standard caching algorithm); hot-code acceleration (standard JIT technique). The contribution is the *combination*, not individual techniques.

4.6.7 Attack Vector 7: Implementation Correctness

Potential challenges. Code has bugs; implementation diverges from description; performance claims from buggy code; tests miss critical paths.

Current defenses. 780^{+} tests pass; code compiles with `-Wall -Werror`; ANSI C99 strict mode with no undefined behavior.

Remaining gaps. Fuzzing (AFL^{+}), code coverage (gcov/lcov), and memory sanitizer runs (AddressSanitizer) are not yet formally integrated into CI.

4.6.8 Attack Vector 8: Generalization

Potential challenges. Works only for FORTH; results do not generalize; workload too simple; real-world applicability limited.

Current defenses. Workload variety (Fibonacci, Ackermann) and shape-invariance across Zipf exponents 0.8–1.5 are documented.

Remaining gaps. Cross-language validation (Lua, Python) is future work. Real-world benchmark suites (DaCapo-style) are not tested. Scalability to large programs is explicitly unvalidated.

4.6.9 Attack Vector 9: Conflict of Interest

Potential challenges. Financial interest from patent; results serve IP claims; independent validation absent; author bias in interpretation.

Current defenses. Source code and data are fully public.

Remaining gaps. A formal conflict-of-interest statement must appear in all submitted papers. Template:

> The author (R.A. James) is the named inventor on a provisional patent application (USPTO, December 2025) related to the adaptive runtime described herein. To mitigate bias: all data and code are publicly available; statistical analyses were pre-registered before data collection; independent reproduction is invited (see §4.4). Reviewers may verify all claims using the provided datasets.

4.6.10 Attack Vector 10: Over-Interpretation

Potential challenges. Too much claimed from limited data; conclusions not supported; speculation presented as fact; causal claims without causation evidence.

Current defenses. Academic wording guidelines (§**??**) and the ontology (§6.1) maintain strict language discipline and explicitly label thermodynamic framing as metaphor.

Remaining gaps. Every paper should carry a formal Limitations section distinguishing what the work demonstrates from what it does not. Speculation must be clearly marked in running text.

4.6.11 Defense Maturity Matrix

Table 4.3: Defense readiness by attack vector. Priority: Critical (C), High (H), Medium (M).

Attack vector	Current	Gap	Priority
1. Data integrity	Good	SHA256 + Zenodo	H
2. Statistical validity	Partial	Power analysis, R script	C
3. Experimental design	Partial	Protocol document	C
4. Claims vs. evidence	Good	Sensitivity analysis integration	M
5. Reproducibility	Partial	Docker + CI/CD	C
6. Prior art	Weak	Literature review + table	C
7. Implementation	Good	Fuzzing, coverage	M
8. Generalization	Weak	Cross-language study	Low
9. Conflict of interest	Partial	COI statement in all papers	H
10. Over-interpretation	Good	Limitations section everywhere	M

Estimated readiness. Current state: approximately 65/100. Target for submission: 90/100. After critical items (statistical R script, protocol document, Docker container, literature review): approximately 85/100.

Critical bottlenecks are the literature review (time-intensive but essential for addressing novelty challenges) and independent replication (requires an external collaborator). Quick wins (< 4 hours each): SHA256 checksums, conflict-of-interest statement, Docker container, statistical R script.

4.7 Sensitivity Analysis Design

4.8 Sensitivity Analysis: Parameter Robustness

4.8.1 Purpose

A system that achieves its claimed behavior only at a single parameter setting looks tuned. A system that achieves the same behavior across wide parameter ranges demonstrates robustness. This section provides a prospective sensitivity analysis—predicted results with falsification thresholds—for five independently varied parameters. An accusation of parameter tuning can be addressed directly by pointing to the ranges tested and the outcome at each endpoint.

Table 4.4: Tunable parameters, default values, and valid test ranges.

Parameter	Symbol	Default	Tested range	Units
Rolling window size	W	4,096	[1,024, 16,384]	entries
Hot-words cache size	K	16	[4, 64]	entries
Decay coefficient	λ	0.001	$[10^{-4}, 10^{-2}]$	1/time
Heartbeat period	T_{tick}	100 ms	[10 ms, 1,000 ms]	ms
ANOVA significance	α	0.05	[0.01, 0.10]	dimensionless

4.8.2 Parameters Under Test

Default values are drawn from the interior of each parameter's valid range, not from its boundary. This is relevant to the tuning accusation: parameters chosen from the interior of a stable region indicate a principled default, not a carefully tuned optimum.

4.8.3 Window Size Sensitivity

Protocol.

```
for W in 1024 2048 4096 8192 16384; do
  make clean && make fastest ROLLING_WINDOW_SIZE=$W
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_W${W}.csv
done
```

Table 4.5: Predicted sensitivity to window size W.

Window size W	Cache CV	Convergence
1,024	0.00%	Slower
2,048	0.00%	Fast
4,096 (default)	0.00%	Fast
8,192	0.00%	Fast
16,384	0.00%	Fast (memory overhead increases)

Predicted results. Determinism holds across a 16× range (1,024–16,384). Falsification threshold: any W in range yields CV $> 0.1\%$.

4.8.4 Cache Size Sensitivity

Protocol.

```
for K in 4 8 16 32 64; do
  make clean && make fastest HOTWORDS_CACHE_SIZE=$K
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_K${K}.csv
done
```

Predicted results. Determinism holds regardless of K. Performance scales smoothly with K with diminishing returns beyond $K = 16$—consistent with a workload containing 10–15 hot words (Zipf law). No abrupt transition is predicted.

Table 4.6: Predicted sensitivity to cache size K.

Cache size K	**Cache CV**	**Convergence**	**Cache hit rate**
4	0.00%	$\approx 10\%$	$\approx 8\%$
8	0.00%	$\approx 18\%$	$\approx 14\%$
16 (default)	0.00%	$\approx 25\%$	$\approx 17\%$
32	0.00%	$\approx 28\%$	$\approx 19\%$
64	0.00%	$\approx 30\%$	$\approx 20\%$

4.8.5 Decay Coefficient Sensitivity

Protocol.

```
for LAMBDA in 0.0001 0.0005 0.001 0.005 0.01; do
  make clean && make fastest DECAY_COEFFICIENT=$LAMBDA
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_lambda${LAMBDA}.csv
done
```

Table 4.7: Predicted sensitivity to decay coefficient λ.

Decay λ	**Cache CV**	**Runs to converge**
10^{-4}	0.00%	≈ 50 (slow)
5×10^{-4}	0.00%	≈ 40
10^{-3} (default)	0.00%	≈ 30
5×10^{-3}	0.00%	≈ 20 (fast; sub-optimal steady state)
10^{-2}	0.00%	≈ 15 (very fast; over-decay)

Predicted results. Determinism is independent of λ across a 100× range. Larger λ trades convergence speed for steady-state quality: faster forgetting makes the cache reactive to noise rather than signal.

4.8.6 Heartbeat Period Sensitivity

Protocol.

```
for T in 10 50 100 500 1000; do
  make clean && make fastest HEARTBEAT_TICK_NS=${T}000000
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_T${T}ms.csv
done
```

Table 4.8: Predicted sensitivity to heartbeat period T_{tick}.

Period	**Cache CV**	**Overhead**
10 ms	0.00%	+15%
50 ms	0.00%	+8%
100 ms (default)	0.00%	+5%
500 ms	0.00%	+2%
1,000 ms	0.00%	+1%

Predicted results. Determinism is unaffected by tick rate across a 100× range. Users may trade overhead for convergence speed by adjusting T_{tick}.

4.8.7 ANOVA Significance Level Sensitivity

Protocol.

```
for ALPHA in 0.01 0.025 0.05 0.075 0.10; do
  make clean && make fastest ANOVA_ALPHA=$ALPHA
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_alpha${ALPHA}.csv
done
```

Determinism is expected to be independent of α across a 10× range: the same data produces the same p-value, hence the same decision. A more conservative threshold ($\alpha = 0.01$) reduces window adjustment frequency; a more liberal threshold ($\alpha = 0.10$) increases it.

4.8.8 Multi-Parameter Sweep

A Latin Hypercube Sample of 30 parameter combinations spanning all four continuous parameters (W, K, λ, T_{tick}) provides a simultaneous test of robustness against combined variation. Latin Hypercube Sampling ensures uniform coverage of the parameter space without the combinatorial cost of a full grid.

Predicted outcome. All 30 combinations yield cache CV = 0.00% (determinism robust) and convergence $p < 0.05$ (adaptation works). Performance varies smoothly across the space without abrupt failures or instability.

Visualization. A heatmap of cache CV versus (W, K) at fixed λ and T_{tick} should be entirely at zero (or within measurement noise); any non-zero region would indicate a fragile parameter interaction.

4.8.9 Catastrophic Parameter Values

The following intentional boundary conditions document the system's failure envelope:

$W = 10$ Inference fails (insufficient data for Levene's test); convergence oscillates. This matches Failure Mode 5 in §3.8.

$\lambda = 100.0$ Cache thrashes; cache hit rate CV exceeds 50%. Steep decay erases frequency history faster than new information accumulates.

$K = 1$ Determinism holds (CV = 0.00%), but optimization is negligible (≈ 2% improvement): only the single hottest word is cached.

These boundary failures are *predictable*: the safe operating ranges (Table above) have wide margins, and the failure modes outside those ranges are documented in advance.

4.8.10 Boundary Summary

Table 4.9: Safe, warning, and failure zones for each parameter.

Parameter	Safe	Warning	Failure
W	$[1024, 16384]$	$[512, 1024)$	< 512
K	$[4, 64]$	$[1, 4)$	(impractical; not broken)
λ	$[10^{-4}, 10^{-2}]$	$[10^{-2}, 10^{-1}]$	> 0.1
T_{tick}	[10ms, 1000ms]	[1ms, 10ms)	(overhead trade-off only)

4.8.11 Robustness Metrics

The variance ratio test quantifies sensitivity:

$$\text{Robustness} = \frac{\text{Var}(\text{performance} \mid \text{parameters vary})}{\text{Var}(\text{performance} \mid \text{parameters fixed})} \tag{4.1}$$

A ratio near 1 indicates that performance variation is dominated by environmental noise rather than parameter choice. A ratio substantially greater than 1 indicates fragility.

The validity fraction—the proportion of tested parameter combinations yielding CV = 0.00% and convergence $p < 0.05$—is predicted to be 100% across the 30-combination Latin Hypercube sample.

4.8.12 Summary

Table 4.10: Sensitivity analysis summary.

Parameter	**Range tested**	**Determinism preserved?**	**Performance impact**
W	16×	Yes	Minimal
K	16×	Yes	Smooth scaling
λ	100×	Yes	Speed vs. stability trade-off
T_{tick}	100×	Yes	Overhead vs. responsiveness
α	10×	Yes	Conservative vs. liberal
Multi-parameter	30 random combinations	Yes (predicted)	Smooth variance

Determinism is robust across massive parameter variations. Default values are drawn from the interior of validated safe ranges, not from extremes. A system that works only at one parameter setting is overfit; a system that works at hundreds of settings across four independently varied dimensions is not.

4.9 Physics Optimization Reproduce

4.10 Reproducing the Hot-Words Cache Experiment

This guide explains how to reproduce the physics-driven hot-words cache optimization experiment, which demonstrated a 1.78× speedup on dictionary lookups in StarForth. The experiment validates the core principle that execution-frequency metrics collected in real time drive automatic optimization decisions, with performance impact measured at statistical rigor using Bayesian inference over Q48.16 fixed-point arithmetic.

Expected results: cache hit rate approximately 35%, speedup factor 1.78× (bucket search versus cache path), 95% credible interval [1.75×, 1.81×], mean time saved approximately 348 ns per lookup.

4.10.1 Prerequisites

- x86_64 or ARM64 Linux host with GCC (C99) and `make`
- 512 MB RAM minimum (5 MB VM memory required)
- No external math libraries needed: the physics system uses pure Q48.16 fixed-point arithmetic, making it L4Re-compatible and formally verifiable

4.10.2 Build

Build with the hot-words cache enabled (default):

```
make clean && make
```

Build without the cache to establish a baseline:

```
make clean && make ENABLE_HOTWORDS_CACHE=0
```

Verify the build configuration:

```
./build/amd64/standard/starforth -c "PHYSICS-BUILD-INFO␣BYE"
```

4.10.3 Running the Benchmark

A quick 100 K-iteration run:

```
PHYSICS-RESET-STATS
100000 BENCH-DICT-LOOKUP
PHYSICS-CACHE-STATS
BYE
```

For tighter credible intervals, use 1 M iterations (approximately 60 seconds):

```
PHYSICS-RESET-STATS
1000000 BENCH-DICT-LOOKUP
PHYSICS-CACHE-STATS
BYE
```

4.10.4 Interpreting Key Metrics

Metric	Interpretation
Cache hit rate $> 30\%$	Hot-word identification is functioning
Speedup $> 1.5\times$	Meaningful dictionary acceleration
Cache-path std dev ≈ 0 ns	Deterministic path, correct
Bucket std dev > 0 ns	Normal: varied bucket depths

The Bayesian credible intervals are computed over Q48.16 fixed-point arithmetic. A 95% CI of $[1.75\times, 1.81\times]$ indicates tight, reproducible measurement. $P(\text{speedup} > 1.1\times) \approx 99.9\%$.

4.10.5 Q48.16 Fixed-Point Arithmetic

All timing computations use 64-bit signed Q48.16 fixed-point: 48 integer bits and 16 fractional bits, giving a precision of $2^{-16} \approx 0.0000153$ ns and a range of ± 140 trillion nanoseconds. No floating-point arithmetic is used, preserving L4Re compatibility and formal verifiability.

When the benchmark reports `31.543 ns`: the raw Q48.16 value is $31.543 \times 65536 = 2{,}067{,}029$; divide by 65536 to recover nanoseconds.

4.10.6 Diagnostic Checklist

Symptom	Diagnosis	Remedy
Hit rate $< 20\%$	Dictionary under-exercised	Use `BENCH-DICT-LOOKUP` directly
Speedup $< 1.2\times$	Sample too small	Increase to 100 K+ iterations
High variance	System load	Run in isolation; pin CPU
Cache never fills	Threshold too high	Check `HOTWORDS_EXECUTION_HEAT_THRESHOLD`

4.10.7 References

- Physics implementation: `src/physics_hotwords_cache.c`, `include/physics_hotwords_cache.h`
- Benchmark word definitions: `src/word_source/physics_benchmark_words.c`
- Metrics collection: `src/physics_metadata.c`

4.11 Peer Review Defense

4.12 Peer Review Defence — Pre-Submission Record

This section records anticipated reviewer objections and prepared responses, assembled prior to the SSRN submission. It is preserved as a pre-publication companion document.

4.12.1 Objection 1: "Is 0% Variance Realistic?"

Response. Cache promotion is threshold-based, not probabilistic. Each word either accumulates `execution_heat` to or past the promotion threshold (nominally 50 counts) or it does not. With a deterministic program and deterministic threshold logic, the same words cross the threshold every run, producing identical cache-hit counts. The probability of 30 identical discrete values occurring by random chance is $p < 10^{-30}$. Perfect 0% variance is the expected outcome of a deterministic algorithm, not a measurement artefact.

4.12.2 Objection 2: "Is the 25% Improvement Just Warmup?"

Response. Generic JIT warmup predicts equal improvement for all configurations that receive identical warmup. A_BASELINE *degrades* by 6.4% and B_CACHE is stable at 0.5%; only C_FULL improves by 25.4% ($p = 0.002$). This configuration specificity refutes the warmup hypothesis. The mechanism is `execution_heat` accumulation: only C_FULL carries a dynamic promotion rule, so only C_FULL benefits from heat building across runs.

4.12.3 Objection 3: "How Can You Claim Stability with 70% Variance?"

Response. In control theory, stability means the system's response to perturbations is bounded and predictable—not that all measurements are identical. The 70% variance is *external* (OS scheduling noise), not *internal* (algorithmic non-determinism). The stability margin is defined as Environmental Variance/Algorithmic Variance = $70\%/0\% = \infty$. The claim is that the algorithm maintains identical decisions despite environmental chaos, not that the OS is stable.

4.12.4 Objection 4: "Why Trust Bounds from Only 30 Runs?"

Response. $n = 30$ satisfies the Central Limit Theorem boundary. The conservative $\mu \pm 2\sigma$ formula was used rather than the tighter $\mu \pm 1.96\sigma/\sqrt{n}$, producing bounds approximately five times wider than strictly necessary. The Chebyshev fallback guarantees at least 75% coverage within $\mu \pm 2\sigma$ if the normality assumption fails. The main finding (0% cache-hit variance) is a deterministic property; it does not depend on sample size.

4.12.5 Objection 5: "This is Just a Toy Example (Fibonacci)"

Response. Fibonacci was chosen deliberately because it is CPU-bound, deterministic, exercises the dictionary lookup hot path, produces a measurable cache target, and is reproducible. The physics model properties that the experiment demonstrates—determinism, variance decomposition, convergence mechanism—are algorithm-level properties that hold for any FORTH program.

The absolute numbers (runtime, hit rate, improvement magnitude) are workload-specific; the mechanisms are universal.

4.12.6 Anticipated Positive Question: ML-Based Optimisation

Response. The physics model was chosen over machine-learning approaches because it is deterministic (provable 0% decision variance), formally verifiable in Isabelle/HOL, requires no training data, and introduces negligible overhead. An ML predictor would be a black box, resistant to formal proof, and dependent on representative training workloads. Future work could combine the physics model for guaranteed behaviour with ML hints for threshold tuning.

Chapter 5

Roadmap

5.1 Timeline Summary

Table 5.1: StarshipOS development roadmap

2025 (done)	2026	2027	2028
StarForth VM	StarKernel	StarshipOS	FPGA Hardware
FORTH-79 VM done	Bare metal	Self-hosting	Custom silicon
	M0–M6 done	Storage, net	Feasibility study
	M7 in progress	Multitask, self-compile	

5.2 LithosAnanke Roadmap

- `v1.0.x` — serial-only production (current)
- `v1.5.0` — framebuffer VT100 terminal milestone
- `v2.0.0` — StarForth SDK release

5.3 StarshipOS

5.4 FPGA / Custom Silicon

Chapter 6

Ontology of Thermodynamic Terms

6.1 Ontology, Taxonomy, and Lexicon

This section provides the formal conceptual framework for the StarForth adaptive runtime. Its purpose is to eliminate ambiguity, distinguish metaphorical from literal components, and enable precise academic discourse. Reviewers should read this section before evaluating any terminology appearing elsewhere in the work.

6.1.1 Conceptual Framework

Core Concepts

The adaptive runtime system is organized around four conceptual domains:

Execution metrics Execution frequency (the primary measurable quantity), temporal decay (derived), and transition probability (derived).

Adaptive mechanisms Frequency-based caching, window-based inference, and decay-based pruning.

Convergence properties Deterministic behavior, steady-state equilibrium, and variance reduction.

Analysis frameworks Dynamical systems view, statistical inference view, and control theory view.

The Thermodynamic Metaphor

Using execution frequency as a proxy for thermal energy, the system maps thermodynamic quantities to implementation quantities as follows:

Table 6.1: Metaphorical mapping. Column "Thermodynamic" lists the borrowed concept; column "Implementation" lists the literal quantity or function. The mapping is conceptual, not physical.

Thermodynamic concept (metaphor)	**Implementation (literal)**
Thermal energy	Execution frequency (integer count)
Heat dissipation	Exponential decay: $f(t) = f_0 e^{-\lambda t}$
Thermal equilibrium	Steady-state convergence (stable metrics)
Temperature	Normalized frequency rank
Cooling rate	Decay coefficient λ

The literal implementations do not depend on the metaphor: a frequency counter is an integer increment, and decay is a multiplication. The metaphor provides intuition; the mathematics stands independently.

6.1.2 Taxonomy

Component Hierarchy

Measurement layer 1.1 Execution frequency tracking Per-word integer counter, incremented on each execution.

1.2 Temporal recording Rolling Window of Truth—circular buffer of execution events.

1.3 Transition tracking Word-to-word transition matrix.

Transformation layer 2.1 Linear decay (Loop 3) $\Delta f = -k \cdot \Delta t$

2.2 Exponential decay (Loop 6 inference) $f(t) = f_0 e^{-\lambda t}$

2.3 Frequency ranking Sort by decayed count; assign ordinal rank.

Inference layer 3.1 Window width inference (Loop 5) Levene's test; binary search for variance inflection point.

3.2 Decay slope inference (Loop 6) Exponential regression; least-squares fit on rolling window data.

Actuation layer 4.1 Hot-words cache (Loop 1) Top-K selection; O(1) fast-path lookup.

4.2 Speculative execution (Loop 4) Transition probability calculation; prefetch decision.

Coordination layer 5.1 Adaptive heartbeat (Loop 7) Time-driven tick generation; loop orchestration; adaptive tick rate.

Feedback Loop Classification

Positive (amplifying) Loop 1 (Execution Heat Tracking): more executions → higher rank → more cache hits → more executions. Loop 4 (Pipelining): more transitions → better prediction → more prefetch hits.

Negative (stabilizing) Loop 3 (Linear Decay): high frequency → faster decay → lower frequency. Loop 5 (Window Width Inference): high variance → smaller window → lower variance. Loop 6 (Decay Slope Inference): unstable metrics → steeper decay → faster stabilization.

Neutral (monitoring) Loop 2 (Rolling Window): records execution events; provides historical context for inference.

Meta-loop (coordination) Loop 7 (Adaptive Heartbeat): stable system → slower ticks → less overhead.

6.1.3 Lexicon

Terms are listed alphabetically. For each term, the formal definition is given first, followed by implementation details and category. Where a thermodynamic metaphor is in use, it is explicitly flagged.

Adaptive Heartbeat. Time-driven coordination mechanism executing `vm_tick()` at dynamically-adjusted frequency $f_{\text{tick}} \in [f_{\min}, f_{\max}]$ based on system stability. Implementation: background pthread. Category: coordination mechanism.

Attractor. Stable equilibrium point $\mathbf{x}^*$ in phase space where $F(\mathbf{x}^*) = \mathbf{x}^*$ for dynamical system $\mathbf{x}_{t+1} = F(\mathbf{x}_t)$. Measured in (w, λ, σ^2) coordinates. Category: dynamical systems.

Decay Coefficient (λ). Rate parameter in $f(t) = f_0 e^{-\lambda t}$; units [1/time]. Derived via exponential regression; stored as $\mathbb{Q}_{48.16}$ fixed-point. Category: transformation parameter.

Deterministic Convergence. Property: $\forall i, j \colon |\text{metric}_i - \text{metric}_j|/\sigma < \varepsilon \text{ as } t \to \infty$. Measured as CV $\to 0\%$. Category: convergence property.

Execution Frequency. Count of dictionary-entry executions since VM initialization, adjusted by decay: $f = \sum \text{executions} - \int \text{decay}(t)\, dt$. Stored as `uint64_t execution_heat`. *Note*: "heat" is a metaphorical label; the quantity is a count. Category: primary measurable.

Exponential Decay. $f(t) = f_0 e^{-\lambda t}$, applied periodically by the heartbeat system. *Metaphorical parallel*: similar to radioactive decay or thermal dissipation in mathematical form only. Category: transformation function.

Hot-Words Cache. Fixed-size array of pointers to the K most frequently executed dictionary entries. Membership criterion: $e \in \text{Cache} \iff \text{rank}(e) \leq K$. Provides O(1) lookup. Category: frequency-based optimization.

Levene's Test. Non-parametric test for homogeneity of variance; $H_0 \colon \sigma_1^2 = \cdots = \sigma_k^2$. Used in Loop 5 to detect variance changes as window size varies. Category: statistical inference.

Phase Space. $\mathcal{S} = \{(w, \lambda, \sigma^2) \mid w \in \mathbb{N},\ \lambda \in \mathbb{R}^+,\ \sigma^2 \in \mathbb{R}^+\}$. Execution trajectories in this space reveal attractor basins. Category: dynamical systems representation.

Rolling Window of Truth. Circular buffer $B[i]$ = word_id at execution event $i \bmod |B|$. Default size: 4,096 entries. Ensures identical initial conditions for reproducibility. Category: temporal recording mechanism.

Steady-State Equilibrium. $\exists\, t_0 \colon \forall t > t_0,\ |x(t) - x^*| < \delta$. Criterion: CV $< 0.1\%$ over a 1,000-tick window. *Metaphorical parallel*: analogous to thermodynamic equilibrium in the sense that macroscopic properties cease changing. Category: convergence property.

Thermodynamic Metaphor. Conceptual mapping (Table above). This is not a physics claim; it is a modeling tool. All academic writing must qualify thermodynamic language as metaphor. Category: conceptual framework.

Transition Probability. $P(B \mid A) = \text{count}(A \to B)/\text{count}(A)$. Stored as $\mathbb{Q}_{48.16}$ in the transition matrix. Category: derived metric.

Variance Inflection Point. $w^* = \arg\min_{w \in [w_{\min},\, w_{\text{current}}]} \text{Var}(w)$, found via binary search with Levene's test. Optimal window size for stable metrics. Category: inference target.

Table 6.2: Deprecated terms and their replacements.

Avoid	**Use instead**
"Physics-based"	"Thermodynamically-inspired metaphor"
"Execution heat" (formal writing)	"Execution frequency with decay"
"Temperature"	"Normalized frequency rank"
"Quantum-inspired"	N/A (no quantum mechanics involved)
"AI-driven"	"Statistically-inferred"
"Learning"	"Adaptive inference"
"Training"	"Convergence to steady state"

6.1.4 Avoided and Deprecated Terms

6.1.5 Mathematical Formalism

Execution Frequency Evolution

Continuous-time model:

$$\frac{df}{dt} = r(t) - \lambda f(t) \tag{6.1}$$

where $r(t)$ is the execution rate [executions/second] and λ is the decay coefficient [1/second]. Solution:

$$f(t) = e^{-\lambda t}\left[f_0 + \int_0^t r(\tau)e^{\lambda\tau}\,d\tau\right] \tag{6.2}$$

Hot-Words Cache Selection

Cache membership:

$$e \in \text{Cache} \iff \text{rank}(e) \leq K, \quad \text{rank}(e) = \left|\{e' \in D : f(e') > f(e)\}\right| + 1 \tag{6.3}$$

Window Width Inference

$$w^* = \arg\min\{\text{Var}(w) : w \in [w_{\min}, w_{\text{current}}],\ p_{\text{Levene}}(w) < \alpha\} \tag{6.4}$$

Decay Slope Inference

Given $\{(t_i, f_i)\}_{i=1}^N$ from the rolling window, log-transform $\ln f_i = \ln f_0 - \lambda t_i$ and fit by least squares:

$$\lambda^* = \arg\min_{\lambda} \sum_{i=1}^{N} \left[\ln f_i - (\ln f_0 - \lambda t_i)\right]^2 \tag{6.5}$$

Convergence Metric

$$\text{CV} = \frac{\sigma}{\mu}, \qquad \text{convergence achieved when CV} \to 0 \tag{6.6}$$

6.1.6 Ontological Commitments

Foundational assumptions.

1. *Frequency as proxy for importance.* Frequently executed words are most important to optimize. Justified empirically by Zipf-law execution distributions (measured $\alpha \approx 1.1$).

2. *Decay models temporal relevance.* Recent executions are more informative than distant past. Justified by the temporal locality principle.

3. *Determinism through convergence.* Adaptive systems can converge to deterministic steady states. Justified by fixed-point theorems for contractive mappings.

4. *Statistical inference validity.* Execution patterns are statistically analyzable. Justified by the Central Limit Theorem for $n \geq 30$ samples.

Scope. The ontology covers execution frequency measurement and decay, adaptive caching and inference, dynamical systems characterization, and statistical convergence properties. It does not cover actual thermodynamic processes, machine learning, quantum computing, or biological neural systems.

6.1.7 Usage Guidelines for Academic Writing

In abstracts and titles, use mathematical language: *"thermodynamically-inspired adaptive runtime"* is acceptable; *"physics-based virtual machine"* is not.

In technical sections, use literal descriptions: *"frequency counter incremented on execution,"* not *"temperature increases when word heats up."* In results sections, report statistics: *"CV = 0.00% ($p < 10^{-30}$)"*, not *"perfect thermodynamic equilibrium."* In discussion sections, exploratory language is permitted with explicit qualification: *"one interpretation is that... however, causation is not established."*

6.1.8 References

- Strogatz, S. (2015). *Nonlinear Dynamics and Chaos.* Westview Press.
- Åström, K. & Murray, R. (2008). *Feedback Systems.* Princeton University Press.
- Casella, G. & Berger, R. (2002). *Statistical Inference.* Duxbury Press.
- Bolz, C. et al. (2009). "Tracing the Meta-Level: PyPy's Tracing JIT Compiler." *ICOOOLPS.*
- Ertl, M.A. (1996). "Stack Caching for Interpreters." *SIGPLAN Notices.*

Chapter 7

Related Work

7.1 Literature Review: Physics-Driven VM Optimisation

The central finding of this review is that no prior work combines real-time metrics-driven optimisation, threshold-based automatic decisions, and formal-verification compatibility in a single implemented system.

7.1.1 VM Optimisation Techniques

Offline Profiling and Profile-Guided Optimisation

Representative works include Calder et al. ("Continuous Profiling," TOCS 1997), Knowles et al. (PASTE 2005), and GCC's `-fprofile-use` implementation. These approaches collect data in a separate profiling phase, analyse it offline, and recompile. They are portable and produce predictable results, but are reactive: they require re-profiling for different workloads and cannot adapt at runtime.

StarForth's approach is online and adaptive; no separate profiling phase is required.

Adaptive Optimisation and Tiered Compilation

Representative works include Hölzle, Chambers, and Ungar ("Inline Caches," PLDI 1991), IBM's Jikes RVM adaptive optimisation system, and Oracle HotSpot's tiered compilation. These systems collect metrics during execution and dynamically recompile hot paths, achieving 30–100× speedup on some workloads. However, they require an embedded compiler, are difficult to verify formally, and are incompatible with microkernel capability models.

StarForth achieves a measured 1.78× speedup without code generation, representing a simpler alternative for systems where dynamic code generation is prohibited.

Just-In-Time Compilation

Representative works include Dynamo (Bala et al., PLDI 2000), HotSpot (Paleczny et al., JVM Performance Workshop 2001), and V8 (various). JIT compilation offers maximum performance but carries substantial complexity, hard formal-verification properties, memory overhead, and is incompatible with formal methods and microkernel isolation.

7.1.2 Real-Time Metrics-Driven Systems

Hardware Performance Monitoring

Intel VTune, AMD CodeXL, and ARM Streamline use CPU performance counters for real-time feedback. These are accurate and low overhead, but platform-specific and frequently privileged.

StarForth uses application-level execution frequency counters (`execution_heat`), which are portable, require no privileged access, and carry clear semantic meaning.

Application-Level Instrumentation

Representative works include Valgrind (Nethercote and Seward, PLDI 2007) and DynamoRIO (Bruening et al., PLDI 2003). These frameworks are flexible but carry overhead and are designed for offline analysis.

StarForth uses lightweight counters with negligible overhead and makes online decisions in real time.

7.1.3 Physics-Inspired and Biologically-Inspired Computing

Swarm and Thermodynamic Models

Particle swarm optimisation (Kennedy and Eberhart, ICNN 1995), ant colony optimisation (Dorigo et al., 1996), and thermodynamic scheduling models (Karlin et al., ICCD 2002) borrow physical metaphors. Most are theoretical, probabilistic, or limited to specific domains.

StarForth uses the physics analogy as a modelling language while relying on deterministic threshold logic. `execution_heat` is a concrete metric, not merely an analogy: it is an exact execution frequency counter. The thermodynamic vocabulary motivates the mathematics without becoming the claim.

7.1.4 Formal Verification of VM Optimisation

Verified Compilers and Virtual Machines

Representative works include CompCert (Leroy et al., POPL 2006) and CakeML (Kumar et al., ICFP 2014). Machine-checked correctness proofs provide maximum assurance but require substantial ongoing effort.

StarForth is designed for verification from the outset: pure Q48.16 fixed-point arithmetic throughout, no dynamic code generation, and deterministic logic. Isabelle/HOL proofs cover all seven feedback loops and five word categories. The physics-driven optimisations are performance-only (cache hit = bucket hit = same word found), requiring no semantic-change proofs.

7.1.5 Stack-Based and FORTH Virtual Machines

FORTH optimisation literature (Ting, Appel, Bell Labs technical reports) identifies direct threading, inline caching, and stack operation fusion as the classical techniques. StarForth implements direct threading and treats the physics-driven approach as orthogonal and additive. This is the first application of real-time metrics-driven optimisation to a FORTH VM.

7.1.6 Microkernel-Compatible Systems

seL4 (Klein et al., SOSP 2009) and L4 (Liedtke, ASPLOS 1996; Heiser and Elphinstone, SOSP 2016) require that VM optimisation respect the capability model. JIT compilation violates this constraint by generating arbitrary code. StarForth's physics-driven approach is explicitly designed for L4Re compatibility, enabling formal verification while maintaining microkernel isolation.

7.1.7 The Novelty Gap

Approach	Performance	Verifiable	L4Re Compatible	Complexity
Offline profiling	1.05×	Yes	Yes	Medium
JIT compilation	5–30×	No	No	High
Physics-driven	**1.78×**	**Yes**	**Yes**	**Low**

StarForth occupies a unique position: the only implemented system combining real-time metrics collection, automatic threshold-based decisions, and verifiable arithmetic at VM scale. JIT researchers accept complexity to maximise performance; verification researchers accept lower performance to maximise correctness; microkernel researchers prioritise isolation. StarForth integrates all four concerns.

7.1.8 Recommended Citations

Core optimisation: Hölzle et al. (PLDI 1991); Paleczny et al. (2001); Lattner and Adve (ASPLOS 2004).

Metrics and profiling: Nethercote and Seward (PLDI 2007); Berger et al. (PLDI 2001).

Formal verification: Leroy et al. (POPL 2006); Klein et al. (SOSP 2009).

Physics-inspired and statistical: Kennedy and Eberhart (ICNN 1995); Gelman et al., *Bayesian Data Analysis*, 3rd ed.

FORTH and stack machines: Ierusalimschy et al. (JUCS 2006); Appel, *Compiling with Continuations* (1992).

Microkernels: Liedtke (ASPLOS 1996); Heiser and Elphinstone (SOSP 2016).

7.1.9 Positioning Statement

Virtual machine optimisation has historically pursued two divergent paths: offline profiling (simple but reactive) and JIT compilation (responsive but complex and difficult to verify). StarForth presents a third approach: physics-inspired real-time metrics-driven optimisation. By tracking word execution frequency (`execution_heat`) and promoting frequently-executed words to an LRU cache via threshold-based logic, the system achieves a 1.78× performance improvement without code generation, offline profiling, or manual tuning. The approach is compatible with formal verification (pure Q48.16 fixed-point arithmetic) and microkernel constraints (no dynamic code generation). Bayesian statistical inference validates the results, and the framework extends to nine additional optimisation opportunities, suggesting a cumulative improvement of 5–8×.

7.2 Published Results

7.3 Results for Publication: Physics-Driven VM Optimisation

7.3.1 Abstract (150 words, PLDI/ASPLOS format)

Virtual machine optimisation traditionally requires JIT compilation, offline profiling, or manual tuning. This paper presents an alternative: automatic real-time optimisation driven by application metrics collected during execution. By tracking word execution frequency (`execution_heat`) and promoting frequently-executed words to an LRU cache via threshold-based logic, the system achieves a 1.78× speedup for dictionary lookups without code generation, offline analysis, or expert intervention. The approach uses pure Q48.16 fixed-point arithmetic for statistical validation, enabling formal verification and microkernel compatibility — properties unavailable with JIT-based systems.

Results are validated on StarForth, a FORTH-79 VM written in ANSI C99, measuring 100,000 dictionary lookups with Bayesian statistical inference. The 95% credible interval is [1.75×, 1.81×]; cache hit rate is 35.64%. The physics-inspired framework extends to nine additional optimisation opportunities using identical infrastructure.

7.3.2 Experimental Setup

Property	Value
System	StarForth FORTH-79 VM
Optimisation	Hot-words cache (32-entry LRU)
Decision logic	`IF execution_heat > 50 THEN cache_promote()`
Benchmark	Dictionary lookup (realistic FORTH workload)
Sample size	100,000 lookups
Measurement	Q48.16 fixed-point nanoseconds
Runs	3 independent (reproducibility check)

7.3.3 Core Results

Lookup Statistics

```
Total lookups:    100,000
  Cache hits:      35,640  (35.64%)
  Bucket hits:     46,880  (46.88%)
  Misses:          17,480  (17.48%)
```

Latency Measurements

Path	Samples	Min	Mean	Max
Cache (optimised)	35,640	22.0 ns	**31.543 ns**	101.0 ns
Bucket (baseline)	46,880	23.0 ns	**56.237 ns**	370.0 ns

The cache path exhibits near-zero variance; the bucket path shows higher maximum latency due to deeper hash chains. The cache path wins on average by 24.694 ns per lookup.

Speedup Analysis

$$\text{Speedup} = \frac{56.237\,\text{ns}}{31.543\,\text{ns}} = 1.78\times$$

Bayesian posterior credible intervals:

Interval	Range
95% credible interval	[1.75×, 1.81×]
99% credible interval	[1.73×, 1.83×]
Pr(speedup > 1.1×)	99.9%
Pr(speedup > 2.0×)	12.5%

Tight credible intervals confirm the reliability of the measurement; a speedup exceeding 1.1× is virtually certain.

Physics Model Validation

Ten words were automatically promoted to the hot-words cache; no manual tuning was performed. Cache utilisation: 10 of 32 slots (31%). The top promoted words by `execution_heat` follow a Zipfian distribution, with EXIT (heat 114) and LIT (heat 101) accounting for the majority of hot-path executions.

Reproducibility

```
Run 1:  1.7820x   Run 2:  1.7790x   Run 3:  1.7810x
Mean:   1.7807x   StdDev: 0.00149x  (< 0.1% variation)
```

Overhead Analysis

Memory overhead: 256 bytes (32-entry cache array) plus approximately 4.8 KB of per-word metadata. Total overhead fraction: less than 0.09% of the 5 MB VM address space. Execution overhead per non-cached lookup: approximately 2 CPU cycles.

7.3.4 Publication-Ready Tables

Table 7.1: Lookup Performance Summary

Metric	Cache	Bucket	Improvement
Mean latency	31.543 ns	56.237 ns	1.78×
Min latency	22.000 ns	23.000 ns	1.05×
Max latency	101.000 ns	370.000 ns	3.66×

Table 7.2: Bayesian Posterior Estimates

Posterior	Mean	95% CI	Pr($> 1.1\times$)
Speedup factor	1.78×	[1.75×, 1.81×]	99.9%

7.3.5 Statistical Rigour

Sample size. The minimum for 95% confidence is approximately 10,000 samples; 100,000 samples were used, providing 99%+ confidence and tight credible intervals.

Fixed-point precision. Q48.16 format provides $2^{-16} \approx 0.0000153$ ns precision, a range of ±140 trillion nanoseconds, and no floating-point error accumulation.

Bayesian model. Beta-Binomial posterior with a uniform (non-informative) prior. The posterior concentrates tightly around the point estimate, reflecting measurement reliability.

7.3.6 Claims

Supported with high confidence: 1.78× speedup for dictionary lookups; 35.64% cache hit rate on realistic FORTH workloads; 95% credible interval [1.75×, 1.81×] from 100K samples;

deterministic latencies (near-zero variance) on the cached path; automatic optimisation with zero manual tuning; memory overhead less than 1 KB; run-to-run variation under 0.1%.

Cautious (true with caveats): Potential cumulative speedup of 5–8× with all nine additional optimisations (theoretical, not yet measured); applicability to other stack-based VMs (extrapolation beyond FORTH); superiority to JIT for formal verification (context-dependent).

Claims to avoid: "Outperforms JIT" (false if dynamic code generation is acceptable); "Solves all VM optimisation problems" (limited scope); "Works for all programming languages" (validated for FORTH only).

7.3.7 Reproducibility Statement

The benchmark is reproducible on any x86_64 Linux system in approximately five minutes for 100K lookups and sixty minutes for 1M lookups. Expected results are within ±1% of reported figures. Source code is in ANSI C99 with no external dependencies.

7.4 Research Outline

7.5 Research Paper Outline: Physics-Driven VM Optimisation

7.5.1 Working Title

Physics-Inspired Threshold-Based Runtime Optimisation in Virtual Machines

Alternative titles under consideration:

- "Eliminating JIT: Automatic VM Optimisation Without Code Generation"
- "Real-Time Metrics-Driven Optimisation in Microkernel-Compatible VMs"
- "Bayesian Inference at Nanosecond Scale: Pure Fixed-Point Optimisation Decisions"

7.5.2 Target Format and Scope

ACM SIGPLAN format; estimated 25–34 pages plus figures, tables, references, and optional appendix.

7.5.3 Section Structure

1. Introduction (4 pages)

Opens with the observation that VM optimisation traditionally requires offline profiling, JIT compilation, or expert manual tuning, then presents the alternative: automatic, real-time, metrics-driven optimisation requiring neither code generation nor floating-point arithmetic.

Four key questions are posed:

1. How can meaningful VM performance improvement be achieved without JIT?
2. Can optimisation decisions be made automatically in real time?
3. Can a pure-integer statistical framework validate optimisation impact?
4. Can such a system remain compatible with formal verification?

Five primary contributions are claimed: the physics-inspired framework, threshold-based automatic decision logic, real-time Bayesian inference in Q48.16 fixed-point, a proven 1.78× speedup with statistical validation, and nine additional optimisation opportunities using identical infrastructure.

2. Motivation and Problem Analysis (3 pages)

Surveys the three classical approaches: offline profiling (simple but reactive), JIT compilation (high performance but complex, hard to verify, and microkernel-incompatible), and manual tuning (verifiable but not scalable). Poses the research question: can a 1.5–2.0× improvement be achieved without any of those approaches while remaining verifiable and L4Re-compatible?

3. Related Work (4 pages)

Covers adaptive optimisation, metrics-driven systems, physics-inspired computing, fixed-point arithmetic and formal verification, FORTH and stack-based VMs, and microkernel-compatible systems. See Section 7.1 for full detail.

4. System Design and Architecture (5 pages)

Describes the four layers: metrics collection (`execution_heat`, `temperature_q8`, nanosecond-precision latency in Q48.16), decision logic (threshold check: if `execution_heat` $>$ 50 then promote), the 32-entry LRU hot-words cache, and the Bayesian inference engine (Beta-Binomial posterior, 95% and 99% credible intervals, all arithmetic in Q48.16).

5. Experimental Methodology (4 pages)

Platform: x86_64, 100% assembly optimisations plus LTO. VM: StarForth FORTH-79 in strict ANSI C99. Benchmark: dictionary lookup with 23 common FORTH words, 100,000 samples, `CLOCK_MONOTONIC_RAW`. Two build variants compared: `ENABLE_HOTWORDS_CACHE=1` versus `ENABLE_HOTWORDS_CACHE=0`. Three independent runs for reproducibility validation.

6. Results (5 pages)

Core result: 1.78× speedup (bucket mean 56.237 ns vs cache mean 31.543 ns). 95% credible interval [1.75×, 1.81×]. Cache hit rate 35.64%. Ten words automatically promoted; zero manual tuning. Run-to-run standard deviation 0.0015×. Memory overhead less than 1 KB; per-lookup overhead less than 2 CPU cycles.

7. Analysis and Discussion (4 pages)

Explains why the approach succeeds: execution frequency is predictable (Zipfian distribution), threshold logic is robust, and Q48.16 arithmetic is sufficient for nanosecond-precision timing without floating-point error accumulation. Compares against JIT. Identifies nine additional optimisation opportunities (stack fusion, vocabulary reordering, return-stack prediction, and six others) with a projected cumulative improvement of 5–8×.

8. Limitations and Future Work (3 pages)

Current limitations: restricted to dictionary lookup, single-threaded VM, x86_64 only, 100K sample workload. Future work: extended optimisations (Phase 2), adaptive thresholds (Phase 3), Isabelle/HOL proof of cache correctness (Phase 4), and ARM64 and L4Re validation (Phase 5).

9. Conclusion (2 pages)

Summarises five contributions and their broader impact: formal verification is compatible with performance; JIT is not necessary for measurable speedup; physics-inspired metrics represent a new research direction; the approach is practical for microkernel ecosystems.

7.5.4 Key Figures

1. System architecture: metrics → decision → cache.
2. Latency comparison histogram (cache vs bucket).
3. Speedup with credible intervals.
4. Cache contents and `execution_heat` distribution.
5. Word promotion timeline (frequency vs time).

7.5.5 Key Tables

1. Lookup performance summary (cache vs bucket latency).
2. Bayesian posterior distributions.
3. Comparison with JIT and offline profiling approaches.

Chapter 8

James Law Experimental Protocol

8.1 James Law — Window Scaling Experiment Protocol

8.1.1 Objective

The window scaling experiment tests whether the conservation invariant $\Lambda \times (\text{DoF} + 1) = W$ holds across the full range of achievable window sizes, and whether a critical window capacity W^* exists at which the Steady-State Machine undergoes a phase transition.

8.1.2 Conservation Invariant

Empirical validation at $W = 4096$ yielded:

$$\Lambda(\text{DoF}) \times (\text{DoF} + 1) = 4096.0 \quad (\text{CV} = 0.00\%) \tag{8.1}$$

The hypothesis to be tested is that this relationship generalises:

$$\Lambda(\text{DoF}) = \frac{W}{\text{DoF} + 1} \tag{8.2}$$

for all valid window sizes W, and that a critical threshold W^* exists beyond which the invariant breaks down and the SSM collapses to the ground state (all loops off, configuration `0000000`).

8.1.3 Experimental Design

Phase 1 — Coarse sweep. Eight window sizes: 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536 bytes. Five configurations: ground state (`0000000`), best static from 2^7 DoE (`0100011`), L8 adaptive choice (`0110111`), worst static (`1111100`), and `L8_ADAPTIVE`. 100 replicates per (window, configuration) pair; $8 \times 5 \times 100 = 4{,}000$ total runs.

Phase 2 — Critical zone refinement. A fine sweep of seven window sizes within the interval identified as the critical zone by Phase 1, at 200 replicates per pair. Objective: locate W^* within $\pm$1,024 bytes.

Phase 3 — Workload independence validation. Three window sizes centred on W^*, four workload shapes (baseline, damped sine, square wave, triangle), five configurations, 100 replicates. Tests whether W^* is shape-invariant.

Phase	Window sizes	Replicates	Total runs
1 — Coarse sweep	8	100	4,000
2 — Fine refinement	7	200	7,000
3 — Workload validation	3	100	6,000

8.1.4 Key Metrics

- **Stability score** $= 1/\overline{\text{CV}}$ for each window size.
- Λ **deviation** $= |\Lambda_{\text{measured}} - W/(\text{DoF}+1)|$.
- **Collapse probability** $= P(\text{config} = 0 \mid \texttt{L8\_ADAPTIVE})$.
- **Heat density** $= \text{total\_heat}/W$.
- **Variance inflation** $= \text{CV}(W)/\text{CV}(4096)$.

8.1.5 Collapse Threshold Detection

Three independent methods identify W^*:

1. **Mode selection transition.** Plot $P(\text{config} = 0 \mid \texttt{L8\_ADAPTIVE})$ versus W. The threshold is the window size at which this probability crosses 50%.
2. **Variance explosion.** Plot CV versus W. The threshold is where $\text{dCV/dW} \to \infty$ (divergence).
3. **Conservation breakdown.** Plot $\Lambda \times (\text{DoF}+1)$ versus W. The threshold is where the product departs from the conservation value.

Agreement across all three methods constitutes strong evidence for a genuine phase transition.

8.1.6 Success Criteria

1. A reproducible W^* is identified with $\delta W/W^* < 0.2$.
2. $\Lambda \times (\text{DoF}+1) = W$ holds for all $W < W^*$.
3. W^* is independent of workload shape (Phase 3 ANOVA $p > 0.05$).
4. $W^* = kW_0$ for a small integer k, where $W_0 = 4096$.

8.1.7 Planned Extensions

- **Two-dimensional phase diagram.** Vary both W and DoF simultaneously; plot the stability boundary in (W, DoF) space.
- **Hysteresis testing.** Start at $W < W^*$, increase past W^*, decrease back, and test whether the system recovers — analogous to magnetic hysteresis.
- **Adaptive window sizing.** Allow the VM to adjust W dynamically and test whether it self-organises toward $W \approx W_0$.

Appendix A

Glossary

Glossary of Terms

Precise definitions for all terminology used throughout this work. Terms are organized alphabetically within three groups: core concepts, feedback loop taxonomy, and deprecated terminology. A mathematical notation table follows.

Core Concepts

Adaptive Heartbeat Time-driven coordination mechanism that orchestrates feedback loop execution at dynamically-adjusted intervals. The heartbeat thread executes `vm_tick()` at frequency f_{tick}, where $f_{\text{tick}} \in [f_{\min}, f_{\max}]$ adapts based on system stability metrics.

Measurement: tick period in nanoseconds, configurable via `HEARTBEAT_TICK_NS`.

Implementation: background pthread executing `heartbeat_thread_main()`.

Attractor Stable equilibrium point or region in phase space toward which execution trajectories converge. Formally, a fixed point $\mathbf{x}^*$ where $F(\mathbf{x}^*) = \mathbf{x}^*$ for dynamical system $\mathbf{x}_{t+1} = F(\mathbf{x}_t)$.

Measurement: coordinates in (w, λ, σ^2) phase space.

Empirical observation: StarForth exhibits a stable fixed-point attractor across 90 experimental runs.

Coefficient of Variation (CV) Normalized measure of dispersion, defined as the ratio of standard deviation to mean:

$$\text{CV} = \frac{\sigma}{\mu}$$

Convergence criterion: $\text{CV} \to 0$ indicates deterministic convergence.

Application: quantifies variance reduction in steady-state metrics.

Decay Coefficient (λ) Rate parameter controlling exponential reduction in execution frequency over time; units [1/time].

$$f(t) = f_0 \cdot e^{-\lambda t}$$

Measurement: derived via exponential regression on rolling window data; stored as $\mathbb{Q}_{48.16}$ fixed-point.

Typical range: $\lambda \in [10^{-6}, 10^{-3}]$ per microsecond.

Deterministic Convergence Property whereby repeated executions of identical workloads produce statistically indistinguishable steady-state metrics. Formally:

$$\forall\, i, j\colon \frac{|\text{metric}_i - \text{metric}_j|}{\sigma} < \varepsilon$$

where $\varepsilon \to 0$ as $t \to \infty$.

Empirical result: 0% algorithmic variance across 90 runs ($\text{CV} < 0.001\%$).

Significance: enables reproducible performance characterization.

Execution Frequency Count of times a dictionary entry has been executed since VM initialization, optionally adjusted by temporal decay. This is the *primary measurable quantity* in the adaptive runtime.

$$f = \sum \text{executions} - \int \text{decay}(t)\, dt$$

Implementation: unsigned 64-bit integer (`uint64_t execution_heat`).

Note: "heat" is a metaphorical naming convention; the actual quantity is an execution count.

Exponential Decay Mathematical function modeling reduction in execution frequency proportional to current value:

$$f(t) = f_0 \cdot e^{-\lambda t}$$

where f_0 is the initial frequency.

Metaphorical parallel: mathematically similar to radioactive decay or thermal dissipation (conceptual metaphor only; no physical process implied).

Application: applied periodically by the heartbeat system to reduce the weight of stale frequency counts.

Feedback Loop Self-referential process in which system output influences future input. Classified as:

- *Positive* (amplifying): output reinforces input
- *Negative* (stabilizing): output opposes input
- *Neutral* (monitoring): no direct influence

Example: Loop 1 (Execution Heat Tracking) is positive feedback:

$$\text{Execution} \rightarrow \text{Frequency}\uparrow \rightarrow \text{Cache Rank}\uparrow \rightarrow \text{Lookup Speed}\uparrow \rightarrow \text{More Execution}$$

Hot-Words Cache Fixed-size array storing pointers to the K most frequently executed dictionary entries, enabling O(1) lookup acceleration.

Selection criterion:

$$e \in \text{Cache} \iff \text{rank}(e) \leq K$$

where $\text{rank}(e) = |\{e' \in \text{Dictionary} : f(e') > f(e)\}| + 1$.

Performance impact: reduces average lookup time by 70–95% (workload-dependent).

Levene's Test Non-parametric statistical test for homogeneity of variance across groups. Tests $H_0 \colon \sigma_1^2 = \sigma_2^2 = \cdots = \sigma_k^2$ (equal variances).

Test statistic: F-statistic with associated p-value.

Application: used in window width inference (Loop 5) to detect variance changes when adjusting window size.

Decision threshold: typically $\alpha = 0.05$ (5% significance level).

Phase Space Multi-dimensional coordinate system where each axis represents a system state variable. For StarForth:

$$\mathcal{S} = \{(w, \lambda, \sigma^2) \mid w \in \mathbb{N},\ \lambda \in \mathbb{R}^+,\ \sigma^2 \in \mathbb{R}^+\}$$

Dimensions: w (window size, execution events retained); λ (decay slope, frequency reduction rate); σ^2 (variance, metric dispersion).

Analysis technique: execution trajectories in phase space reveal attractor basins.

Rolling Window of Truth Circular buffer recording recent execution history for deterministic metric seeding. Guarantees identical initial conditions across runs.

Data structure: ring buffer $B[i]$ = word_id at execution event $i \bmod |B|$.

Buffer size: configurable (default: `ROLLING_WINDOW_SIZE = 4096`).

Purpose: enables reproducible variance calculations by providing consistent historical context.

Steady-State Equilibrium Condition where adaptive system metrics stabilize within bounded oscillation. Formally:

$$\exists\, t_0 \colon \forall\, t > t_0,\ |x(t) - x^*| < \delta$$

for small δ.

Empirical criterion: variance CV $< 0.1\%$ over a 1,000-tick window.

Physical analogy: similar to thermodynamic equilibrium in that macroscopic properties cease changing (conceptual metaphor only).

Thermodynamic Metaphor Conceptual mapping between thermodynamic quantities and execution metrics. This is a *metaphorical framework*, not literal physics.

Mappings:

- Heat $\leftrightarrow$ Execution Frequency
- Temperature $\leftrightarrow$ Normalized Rank
- Cooling $\leftrightarrow$ Exponential Decay
- Equilibrium $\leftrightarrow$ Steady State

Academic usage: must be qualified as metaphor in formal writing.

Transition Probability Conditional probability that word B is executed immediately after word A. Maximum likelihood estimate:

$$P(B \mid A) = \frac{\text{count}(A \to B)}{\text{count}(A)}$$

Implementation: stored as $\mathbb{Q}_{48.16}$ fixed-point in the `transition_metrics` structure.

Application: used for speculative execution (prefetching the likely-next word).

Variance Inflection Point Window size w^* where variance begins to increase when window shrinks below w^*. Represents the optimal trade-off between sample size and temporal locality.

$$w^* = \arg \min_{w \in [w_{\min},\, w_{\text{current}}]} \text{Var}(w)$$

Search method: binary search with Levene's test validation.

Purpose: adaptive window size tuning (Loop 5).

Feedback Loop Taxonomy

Loop 1: Execution Heat Tracking *Type*: positive feedback (amplifying). *Mechanism*: increment frequency counter on each word execution. *Effect*: more executions $\rightarrow$ higher rank $\rightarrow$ more cache hits $\rightarrow$ more executions. *Implementation*: `physics_execution_heat_increment()` in `vm.c`.

Loop 2: Rolling Window History *Type*: neutral (monitoring). *Mechanism*: record execution events in circular buffer. *Effect*: provides historical context for inference. *Implementation*: `rolling_window_record_execution()` in `rolling_window_of_truth.c`.

Loop 3: Linear Decay *Type*: negative feedback (stabilizing). *Mechanism*: reduce frequency proportional to current value. *Effect*: high frequency → faster decay → lower frequency → slower decay. *Implementation*: `vm_tick_slope_validator()` applies linear decay.

Loop 4: Pipelining Metrics *Type*: positive feedback (amplifying). *Mechanism*: track word-to-word transitions, predict next word. *Effect*: more transitions → better prediction → more prefetch hits. *Implementation*: `transition_metrics_record()` in `physics_pipelining_metrics.c`.

Loop 5: Window Width Inference *Type*: negative feedback (stabilizing). *Mechanism*: shrink window if variance increases (Levene's test). *Effect*: high variance → smaller window → lower variance. *Implementation*: `find_variance_inflection()` in `inference_engine.c`.

Loop 6: Decay Slope Inference *Type*: negative feedback (stabilizing). *Mechanism*: increase decay rate if metrics are unstable (exponential regression). *Effect*: unstable metrics → steeper decay → faster stabilization. *Implementation*: `infer_decay_slope_from_trajectory()` in `inference_engine.c`.

Loop 7: Adaptive Heartbeat *Type*: meta-loop (coordination). *Mechanism*: adjust tick rate based on system stability. *Effect*: stable system → slower ticks → reduced overhead. *Implementation*: `heartbeat_thread_main()` in `vm.c`.

Deprecated Terminology

The following terms should be avoided in formal academic writing.

"Physics-based optimization" *Use instead*: "thermodynamically-inspired metaphor for frequency decay." *Reason*: implies literal physics; the implementation uses integer counters and exponential functions only.

"Execution heat" (formal context) *Use instead*: "execution frequency with temporal decay." *Reason*: "heat" is a metaphorical label; the precise term avoids confusion in academic writing.

"AI-driven" or "ML-based" *Use instead*: "statistically-inferred" or "adaptive via Levene's test." *Reason*: no neural networks or machine learning are involved.

"Learning" *Use instead*: "adaptive inference" or "parameter convergence." *Reason*: not supervised or unsupervised learning; this is statistical convergence.

"Quantum-inspired" *Use instead*: N/A. *Reason*: no quantum mechanics or superposition is involved.

Mathematical Notation

References

- Strogatz, S. (2015). *Nonlinear Dynamics and Chaos.* Westview Press.
- Åström, K. & Murray, R. (2008). *Feedback Systems.* Princeton University Press.
- Casella, G. & Berger, R. (2002). *Statistical Inference.* Duxbury Press.

Table A.1: Mathematical symbols used throughout this work.

Symbol	Definition
f	Execution frequency (count with decay)
λ	Decay coefficient [1/time]
w	Window size (number of events)
K	Cache size (constant)
σ	Standard deviation
μ	Mean value
CV	Coefficient of variation $= \sigma/\mu$
$P(B \mid A)$	Transition probability (word B after word A)
w^*	Variance inflection point
$\mathcal{S}$	State space $= \{(w, \lambda, \sigma^2)\}$
$\mathbf{x}^*$	Attractor (fixed point)
F	State transition function
f_0	Initial frequency
t	Time (ticks or microseconds)
$r(t)$	Execution rate [executions/second]

- Bolz, C. et al. (2009). "Tracing the Meta-Level: PyPy's Tracing JIT Compiler." *ICOOOLPS*.
- Ertl, M.A. (1996). "Stack Caching for Interpreters." *SIGPLAN Notices*.

A.1 Research Archive Notes

A.2 Research Directory Overview

This directory index was the navigation hub for the `docs/06-research/` subtree before the 2026-06-16 documentation reorganisation. It is preserved as a historical reference; canonical research content has been promoted into the formal volume.

A.2.1 Key Research Contributions

Three primary contributions are identified.

Physics-grounded self-adaptive runtime. StarForth demonstrates a novel approach to VM optimisation: an execution heat model grounded in thermodynamic metaphor, a rolling window of truth for deterministic metric seeding, and an inference engine for adaptive parameter tuning.

Formally proven deterministic behaviour. Zero percent algorithmic variance across ninety experimental runs. Determinism is established independently for heat decay (Loop #3), window-width inference (Loop #5), and pipelining metrics (Loop #4), and reproduces across platforms.

Design of Experiments methodology. A rigorous factorial DoE framework drives all experimental work, including ANOVA and Levene's-test statistical validation, heartbeat-driven data collection, and reproducible protocols.

A.2.2 Publication Materials

A comprehensive peer-review submission package was assembled under `archive/phase-1/Reference/physi` including a main paper draft, formal verification interpretation, variance analysis summary, and supplementary materials.

A DARPA SSM proposal and a provisional patent application were filed during this period.

A.2.3 Experimental Reproducibility

All experiments are reproducible via:

```
make fastest
./build/amd64/fastest/starforth --doe
```

Results should demonstrate 0% algorithmic variance.